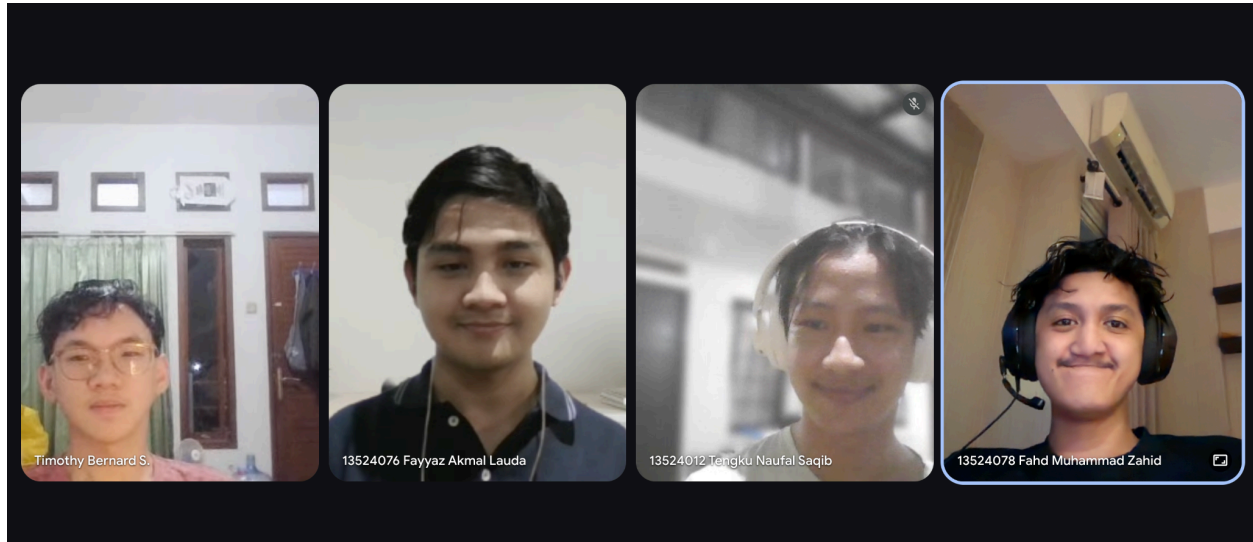


Laporan Tugas Besar
IF2224 Teori Bahasa Formal dan Otomata
Milestone 3 : Semantics Analysis



Disusun oleh Kelompok :

Fahd Muhammad Zahid	13524078
Timothy Bernard Soeharto	13524092
Fayyaz Akmal Lauda	13524076
Tengku Naufal Saqib	13524012

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG 2026

Bab I : Landasan Teori

1.1 Analisis Semantik

Dalam proses kompilasi, analisis semantik merupakan tahap ketiga setelah analisis leksikal dan analisis sintaksis. Analisis leksikal bertugas memecah kode sumber menjadi token, analisis sintaksis memeriksa apakah urutan token membentuk struktur yang valid sesuai grammar, sedangkan analisis semantik memeriksa apakah program yang secara sintaksis valid juga bermakna secara logika.

Analisis semantik tidak dapat dilakukan hanya dengan melihat struktur pohon secara lokal, ia membutuhkan informasi konteks yang lebih luas, seperti apakah suatu *identifier* sudah dideklarasikan sebelum digunakan, apakah tipe data pada kedua sisi *assignment* kompatibel, atau apakah jumlah argumen yang diberikan ke suatu prosedur sesuai dengan deklarasinya. Kesalahan yang ditemukan pada tahap ini disebut *semantic error*, berbeda dari *syntax error* yang ditemukan pada tahap sebelumnya.

Pada implementasi kompilator bahasa Arion, analisis semantik menghasilkan dua output utama: *Decorated AST* (AST yang sudah dianotasi dengan informasi tipe dan posisi di tabel simbol) dan *Symbol Table* yang berisi seluruh informasi identifier yang dideklarasikan dalam program. Kedua output ini menjadi masukan untuk tahap selanjutnya, yaitu pembangkitan kode (*code generation*).

1.2 Abstract Syntax Tree (AST)

Parse Tree atau *Concrete Syntax Tree* adalah representasi langsung dari struktur grammar, setiap aturan produksi yang dipakai parser tecermin sebagai node dalam pohon. Akibatnya, parse tree seringkali memiliki banyak node perantara yang tidak membawa informasi semantik, seperti node untuk tanda kurung, titik koma, atau keyword yang hanya berfungsi sebagai pemisah sintatik.

Abstract Syntax Tree (AST) adalah representasi yang lebih ringkas dan lebih dekat dengan makna program. Node-node yang tidak relevan secara semantik dihilangkan, dan struktur pohon langsung mencerminkan hubungan logis antar konstruksi program. Sebagai contoh, ekspresi $(a + b) * c$ dalam parse tree akan memiliki node untuk kedua tanda kurung, sedangkan dalam AST tanda kurung tersebut sudah tidak ada karena urutan operasi sudah tercermin dalam struktur pohon itu sendiri.

Proses konversi dari *parse tree* ke AST disebut *tree transformation*. Pada implementasi ini, konversi dilakukan oleh class *ASTBuilder* yang menelusuri *ParseTreeNode* secara rekursif dan merakitnya menjadi node-node AST yang sesuai. Setiap jenis konstruksi bahasa direpresentasikan oleh *subclass* tersendiri yang mewarisi dari *base class ASTNode*.

1.3 Symbol Table

Tabel simbol (*symbol table*) adalah struktur data yang digunakan kompilator untuk menyimpan informasi tentang setiap *identifier* yang dideklarasikan dalam program. Informasi ini dibutuhkan baik saat analisis semantik maupun saat pembangkitan kode.

Pada implementasi kompilator bahasa Arion, tabel simbol terdiri dari tiga tabel terpisah yang saling merujuk:

1.3.1 Tabel Identifier (**tab**)

Menyimpan satu entri untuk setiap *identifier* yang dideklarasikan. Setiap entri menyimpan: nama *identifier* (*id*), kelas objeknya (*obj*: CONSTANT, VARIABLE, TYPE, PROCEDURE, atau FUNCTION), tipe datanya (*type*), lexical level tempat ia dideklarasikan (*lev*), alamat atau *offset* memorinya (*adr*), indeks ke tabel lain jika tipenya komposit (*ref*), dan penanda apakah ia parameter *by-reference* (*nrm*). Field link digunakan untuk membentuk *linked list* per blok, menghubungkan *identifier* satu ke *identifier* sebelumnya dalam *scope* yang sama.

1.3.2 Tabel Blok (**btab**)

Menyimpan satu entri untuk setiap blok dalam program blok global, blok tiap prosedur, blok tiap fungsi, dan blok tiap record. Setiap entri menyimpan indeks tab dari identifier terakhir yang dideklarasikan di blok ini (*last*), indeks tab dari parameter terakhir (*lpar*), total ukuran parameter (*psize*), dan total ukuran variabel lokal (*vsize*).

1.3.3 Tabel Array (**atab**)

Menyimpan satu entri untuk setiap tipe *array* yang dideklarasikan. Setiap entri menyimpan tipe index (*xtyp*), tipe elemen (*etyp*), indeks ke atab/btab jika elemen bersifat komposit (*eref*), batas bawah dan atas *index* (*low*, *high*), ukuran satu elemen (*elsz*), dan total ukuran *array* (*size*).

1.4 *Management Scope and Lexical Level*

Scope adalah wilayah dalam program di mana suatu *identifier* berlaku dan dapat diakses. Bahasa Pascal dan turunannya, termasuk Arion, menggunakan *static scoping* (juga disebut *lexical scoping*), artinya *scope* suatu *identifier* ditentukan pada saat kompilasi berdasarkan struktur teks program, bukan pada saat eksekusi.

Setiap blok dalam program, baik blok global, blok prosedur, maupun blok fungsi, membentuk satu *scope* tersendiri. *Identifier* yang dideklarasikan di blok dalam (*inner scope*) bersifat lokal dan tidak terlihat dari luar. Sebaliknya, *identifier* di blok luar (*outer scope*) tetap dapat diakses dari blok dalam, kecuali jika ada *identifier* dengan nama yang sama dideklarasikan ulang di blok dalam (*shadowing*).

Untuk mengimplementasikan ini, digunakan konsep *lexical level* blok global berada di *level* 0, prosedur atau fungsi yang dideklarasikan di *level global* berada di *level* 1, dan seterusnya untuk

setiap tingkat *nesting*. Saat proses lookup *identifier*, pencarian dimulai dari level saat ini dan naik ke level yang lebih rendah hingga *identifier* ditemukan atau seluruh *level* sudah ditelusuri.

Pada implementasi ini, manajemen *scope* dilakukan melalui *scope_stack* di dalam class *SymbolTable*. Saat memasuki blok baru, *open_scope()* dipanggil untuk membuat entri btab baru dan mendorongnya ke stack. Saat keluar dari blok, *close_scope()* dipanggil untuk memopnya.

1.5 *Predefined Identifiers*

Bahasa Arion menyediakan sejumlah *identifier* yang sudah tersedia tanpa perlu dideklarasikan oleh programmer, karena mereka merupakan bagian dari bahasa itu sendiri. *Identifier-identifier* ini diinisialisasi di *level 0* pada saat konstruksi tabel simbol, sebelum program pengguna mulai diproses.

Predefined identifiers yang tersedia meliputi tipe dasar (*integer*, *real*, *char*, *boolean*, *string*), konstanta boolean (*true*, *false*), serta prosedur dan fungsi bawaan (*write*, *writeln*, *read*, *readln*). Keberadaan *identifier* ini di tabel simbol memungkinkan *semantic visitor* mengenali dan memvalidasi penggunaannya dengan mekanisme yang sama seperti *identifier* yang dideklarasikan pengguna.

1.6 *Type Checking*

Pengecekan tipe (*type checking*) adalah proses memverifikasi bahwa operasi-operasi dalam program diterapkan pada operand dengan tipe yang sesuai. Terdapat dua pendekatan utama *static type checking* yang dilakukan saat kompilasi, dan *dynamic type checking* yang dilakukan saat *runtime*. Bahasa Arion menggunakan *static type checking*.

Terdapat dua konsep kompatibilitas tipe yang perlu dibedakan:

Type Compatibility mengacu pada apakah dua tipe dapat digunakan bersama dalam suatu operasi. Misalnya, *integer* dan *real compatible* dalam operasi aritmatika karena *integer* dapat dipromosikan ke *real* secara implisit.

Assignment Compatibility mengacu pada apakah nilai bertipe tertentu dapat di-assign ke variabel bertipe lain. Aturannya lebih ketat: *real* tidak dapat di-assign ke variabel *integer* tanpa konversi eksplisit, meskipun keduanya *compatible* dalam operasi aritmatika.

Aturan *type checking* yang diterapkan pada bahasa Arion antara lain :

- Operasi aritmatika (+, -, *, /) menghasilkan *real* jika salah satu operand *real*, dan *integer* jika keduanya *integer*
- Operasi *div* dan *mod* hanya berlaku untuk *integer*
- Operasi relasional (<, >, <=, >=, ==, <>) menghasilkan *boolean*

- Operasi logika (and, or, not) hanya berlaku untuk boolean dan menghasilkan boolean
- Kondisi pada if, while, dan repeat harus bertipe boolean
- Variabel kontrol pada for harus bertipe ordinal (integer atau char)
- Tipe index array tidak boleh real
- Batas subrange tidak boleh real dan nilai bawah harus lebih kecil atau sama dengan nilai atas

1.7 Decorator AST (*Annotated AST*)

Setelah analisis semantik selesai, setiap node AST dianotasi dengan informasi yang diperoleh selama proses tersebut. AST yang sudah dianotasi ini disebut *Decorated AST* atau *Annotated AST*.

Anotasi yang ditambahkan pada setiap node meliputi :

- `type_code`, kode numerik tipe data node tersebut
- `tab_index`, indeks ke tabel simbol untuk identifier yang berkaitan
- `lev`, lexical level di mana identifier dideklarasikan. Informasi ini sangat penting untuk tahap pembangunan kode

Karena dari sini kompilator mengetahui ukuran memori yang dibutuhkan, cara mengakses variabel berdasarkan levelnya, dan instruksi mesin yang tepat untuk tiap operasi berdasarkan tipe datanya.

1.8 Visitor Pattern

Visitor Pattern adalah salah satu behavioral *design pattern* yang memisahkan algoritma dari struktur objek yang dioperasikannya. Dalam konteks AST, struktur objeknya adalah hirarki node AST, sedangkan algoritmanya adalah semantic analysis.

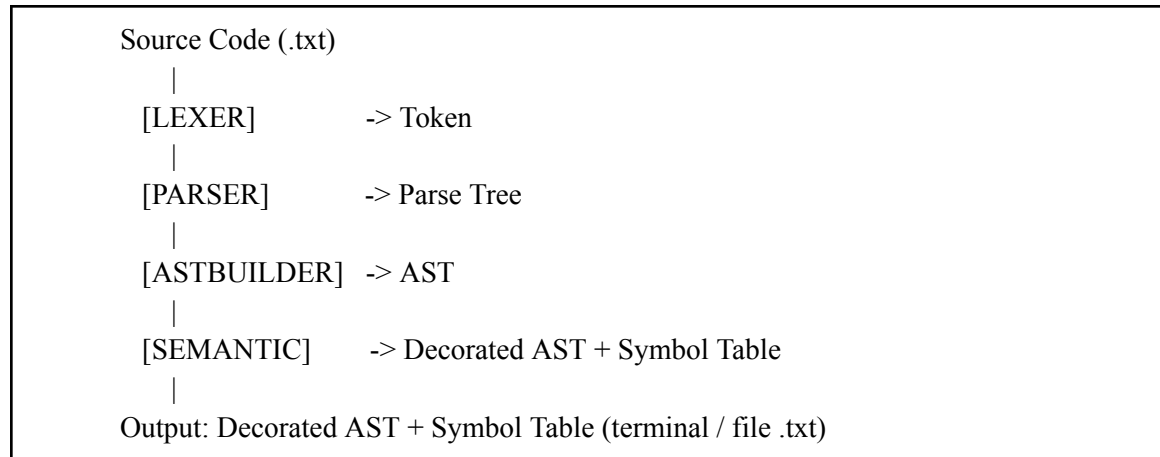
Tanpa *Visitor Pattern*, setiap operasi baru pada AST (misalnya *type checking*, *constant folding*, atau *code generation*) harus menambahkan *method* baru ke setiap *class* node, yang melanggar prinsip *Open/Closed Principle*. Dengan *Visitor Pattern*, operasi baru cukup diimplementasikan sebagai *class* visitor baru tanpa mengubah *class* node sama sekali.

Pada implementasi ini, *class* `SemanticVisitor` berperan sebagai visitor dengan satu *method* `visit_<NamaNode>()` untuk setiap jenis node AST. *Method* `visit()` bertindak sebagai *dispatcher* yang meneruskan panggilan ke *method* yang sesuai berdasarkan tipe dinamis node menggunakan `dynamic_cast`.

Bab II : Perancangan dan Implementasi

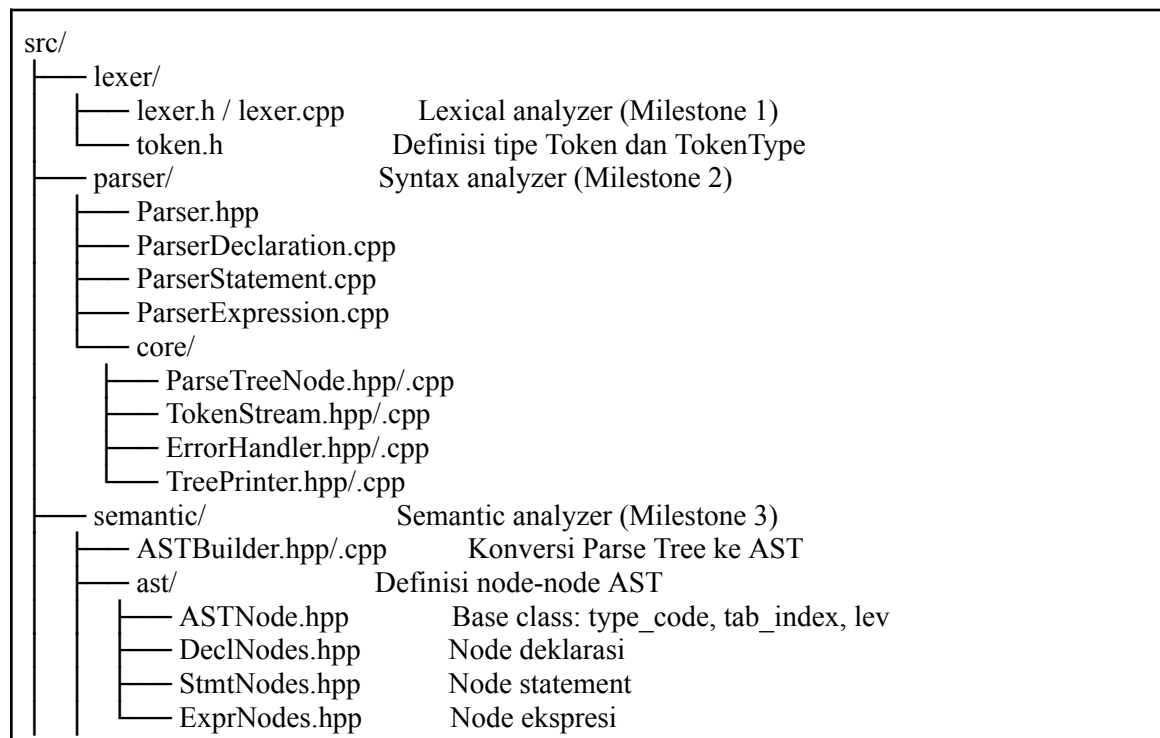
2.1 Gambaran Umum Arsitektur

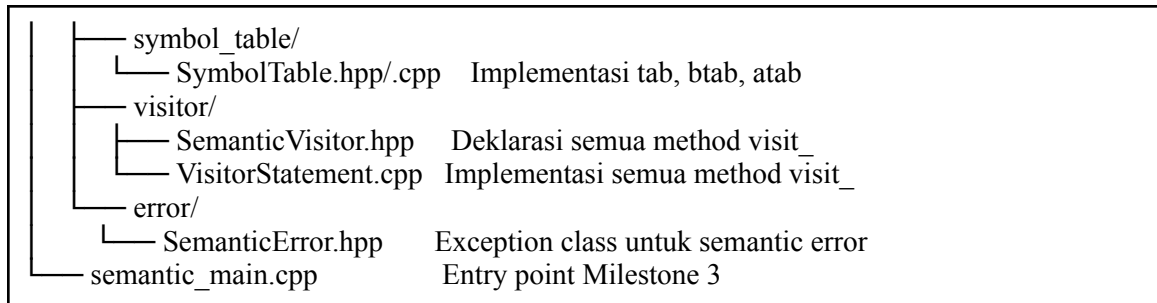
Program semantic analyzer Arion diimplementasikan sebagai tahap ketiga dalam pipeline kompilasi. Program menerima source code Arion sebagai input tunggal, kemudian memprosesnya secara berurutan melalui empat komponen utama yang dijalankan dalam satu proses.



2.2 Struktur Direktori

Program disusun dengan struktur direktori yang mencerminkan pembagian tanggung jawab setiap komponen. Milestone 3 menambahkan folder semantic ke dalam struktur yang sudah ada dari Milestone 1 dan 2.





2.3 AST Node Classes

AST direpresentasikan sebagai hierarki class yang semuanya mewarisi dari ASTNode. Base class ini menyimpan atribut umum yang dibutuhkan pada semantic analysis, yaitu:

- type_code: tipe data node
- tab_index: indeks ke symbol table
- lev: lexical level
- line: nomor baris untuk error reporting

Node AST dibagi menjadi tiga kelompok utama:

- Declaration nodes digunakan untuk deklarasi program, variabel, konstanta, tipe, prosedur, dan fungsi.
- Statement nodes digunakan untuk instruksi seperti assignment, if, while, for, repeat, case, dan procedure call.
- Expression nodes digunakan untuk ekspresi seperti operasi biner, operasi unary, variabel, literal, akses array, akses record, dan function call.

Dengan struktur ini, AST menjadi lebih ringkas daripada parse tree karena tidak lagi menyimpan detail sintaksis seperti semicolon, beginsy, endsy, atau node grammar perantara yang tidak diperlukan pada tahap semantic analysis.

2.4 ASTBuilder

ASTBuilder bertugas mengubah ParseTreeNode hasil parser menjadi AST. Proses ini dilakukan secara rekursif melalui method build_*, misalnya:

- build_program() membentuk ProgramNode
- build_var_declaration() membentuk VarDeclNode
- build_assignment_statement() membentuk AssignNode
- build_expression() membentuk node ekspresi

Pada bagian statement, ASTBuilder mengubah struktur parse tree menjadi node yang lebih langsung merepresentasikan makna program. Assignment statement menjadi AssignNode, percabangan menjadi IfNode, perulangan menjadi WhileNode, ForNode, atau RepeatNode, dan pemanggilan prosedur menjadi ProcCallNode.

Pada bagian ekspresi, ASTBuilder tetap menjaga prioritas operator. build_expression() menangani operator relasional, build_simple_expression() menangani operator seperti +, -, dan or,

sedangkan `build_term()` menangani operator seperti `*`, `/`, `div`, `mod`, dan `and`. Contohnya, ekspresi `a + b * 2` dibentuk menjadi `BinOpNode("+")`, dengan operand kanan berupa `BinOpNode("*")`.

ASTBuilder juga memberi anotasi tipe awal untuk literal:

- `intcon` menjadi `INTEGER`
- `realcon` menjadi `REAL`
- `charcon` menjadi `CHAR`
- `string` menjadi `STRING`
- `true` dan `false` menjadi `BOOLEAN`

Anotasi lanjutan seperti pengecekan tipe, lookup identifier, dan pengisian `tab_index` dilakukan pada tahap berikutnya oleh `SemanticVisitor`.

2.5

Semantic Action

Tabel berikut adalah semantic action yang dijalankan oleh ASTBuilder untuk setiap produksi:

Production Rule	Semantic Action
<code><program> -> <program-header> <decl-part> <compound-statement> .</code>	<code>program.node = new ProgramNode(name=ID, decls=decl.list, body=block)</code>
<code><var-declaration> -> var <id-list> : <type> ;</code>	<code>decl.node = new VarDeclNode(names=id.list, type_node=type.node)</code>
<code><const-declaration> -> const ID == <value> ;</code>	<code>decl.node = new ConstDeclNode(name=ID, value=val.node)</code>
<code><type-declaration> -> type ID == <type-def> ;</code>	<code>decl.node = new TypeDeclNode(name=ID, type_def=type.node)</code>
<code><procedure-decl> -> procedure ID (<params>) ; <body></code>	<code>decl.node = new ProcDeclNode(name=ID, params=p.list, body=b.node)</code>
<code><function-decl> -> function ID (<params>) : <type> ; <body></code>	<code>decl.node = new FuncDeclNode(name=ID, params=p.list, return_type=t.node, body=b.node)</code>
<code><assignment-statement> -> <var> := <expr></code>	<code>stmt.node = new AssignNode(target=v.node, value=e.node)</code>
<code><if-statement> -> if <expr> then <stmt> else <stmt></code>	<code>stmt.node = new IfNode(cond=e.node, then=t.node, else=el.node)</code>
<code><while-statement> -> while <expr> do <compound></code>	<code>stmt.node = new WhileNode(cond=e.node, body=c.node)</code>
<code><for-statement> -> for ID := <e1> to/downto <e2> do <compound></code>	<code>stmt.node = new ForNode(var=ID, from=e1, to=e2, is_downto=flag, body=c)</code>
<code><repeat-statement> -> repeat <stmts> until <expr></code>	<code>stmt.node = new RepeatNode(body=stmts, cond=e.node)</code>

Production Rule	Semantic Action
<case-statement> -> case <expr> of <branches> end	stmt.node = new CaseNode(selector=e, cases=branches.list)
<proc-call> -> ID (<params>)	stmt.node = new ProcCallNode(name=ID, args=params.list)
<expression> -> <simple-expr> rel-op <simple-expr>	e.node = new BinOpNode(op, lhs, rhs)
<simple-expr> -> <simple-expr> <add-op> <term>	e.node = new BinOpNode(op, lhs, rhs)
<simple-expr> -> - <term>	e.node = new UnaryOpNode('-', t.node)
<term> -> <term> <mul-op> <factor>	t.node = new BinOpNode(op, lhs, rhs)
<factor> -> not <factor>	f.node = new UnaryOpNode('not', sub.node)
<factor> -> INTCON	f.node = new NumberNode(value=INTCON.val)
<factor> -> REALCON	f.node = new RealNode(value=REALCON.val)
<factor> -> CHARCON	f.node = new CharNode(value=CHARCON.val)
<factor> -> STRING	f.node = new StringNode(value=STRING.val)
<factor> -> (<expr>)	f.node = e.node (kurung dibuang)
<variable> -> ID	v.node = new VarNode(name=ID)
<variable> -> <var> [<expr>]	v.node = new ArrayAccessNode(array=v, indices=[e])
<variable> -> <var> . ID	v.node = new RecordAccessNode(record=v, field=ID)
<array-type> -> array [<range>] of <type>	t.node = new ArrayTypeNode(index=r.node, elem=t.node)
<record-type> -> record <field-list> end	t.node = new RecordTypeNode(fields=f.list)
<range> -> <const> .. <const>	r.node = new SubrangeNode(low=lo, high=hi)

2.6

Symbol Table

SymbolTable diimplementasikan dengan tiga vector internal: tab, btab, dan atab. Indeks 0-37 diisi pada inisialisasi dengan reserved words (AND..WHILE), tipe dasar (integer, real, char, boolean, string), konstanta boolean (true, false), dan predefined procedures (writeln, readln, write, read). Identifier user dimulai dari indeks 38.

Method utama SymbolTable:

- open_scope(): membuat entri btab baru dan mendorongnya ke scope_stack

- `close_scope()`: memop `scope_stack` saat keluar dari blok
- `enter_tab(entry)`: mendaftarkan identifier baru, otomatis mengatur link dan lev
- `lookup(name)`: mencari identifier dari scope terdalam ke terluar
- `lookup_in_level(name, level)`: mencari hanya di level tertentu untuk deteksi duplikat

2.7 Predefined Identifier yang Digunakan

Pada konstruktor `SymbolTable()`, prosedur `init_predefined()` mengisi tab dengan reserved word, tipe dasar, konstanta boolean, dan prosedur I/O. Indeks aktual hasil eksekusi `./semantic parse_tree_tc1.txt` (parse tree dihasilkan lebih dulu oleh `./parser`):

Idx	Identifier	Kategori	Tipe
0	and	RESERVED	VOID
1	array	RESERVED	VOID
2	begin	RESERVED	VOID
3	case	RESERVED	VOID
4	const	RESERVED	VOID
5	div	RESERVED	VOID
6	downto	RESERVED	VOID
7	do	RESERVED	VOID
8	else	RESERVED	VOID
9	end	RESERVED	VOID
10	for	RESERVED	VOID
11	function	RESERVED	VOID
12	if	RESERVED	VOID
13	mod	RESERVED	VOID
14	not	RESERVED	VOID
15	of	RESERVED	VOID
16	or	RESERVED	VOID
17	procedure	RESERVED	VOID
18	program	RESERVED	VOID
19	record	RESERVED	VOID
20	repeat	RESERVED	VOID
21	integer	TYPE	INTEGER
22	real	TYPE	REAL
23	boolean	TYPE	BOOLEAN

24	char	TYPE	CHAR
25	string	TYPE	STRING
26	then	RESERVED	VOID
27	to	RESERVED	VOID
28	type	RESERVED	VOID
29	until	RESERVED	VOID
30	var	RESERVED	VOID
31	while	RESERVED	VOID
32	true	CONSTANT	BOOLEAN
33	false	CONSTANT	BOOLEAN
34	writeln	PROCEDURE	VOID
35	readln	PROCEDURE	VOID
36	write	PROCEDURE	VOID
37	read	PROCEDURE	VOID

2.8 Semantic Visitor

SemanticVisitor mengimplementasikan Visitor Pattern dengan satu method `visit_<NamaNode>()` untuk setiap jenis node AST. Method `visit()` bertindak sebagai dispatcher menggunakan `dynamic_cast`. Setiap `visit` method mengikuti pola empat langkah:

1. Visit anak-anak node terlebih dahulu untuk mendapatkan atribut semantiknya dari bawah ke atas (synthesized attributes)
2. Lakukan pengecekan dan kalkulasi berdasarkan hasil visit anak
3. Lempar `SemanticError` jika ada pelanggaran aturan semantik
4. Anotasi node saat ini dengan mengisi `type_code`, `tab_index`, dan `lev`

Pengecekan yang dilakukan pada setiap konstruksi bahasa adalah sebagai berikut:

- Literal seperti number, real, char, string, dan bool hanya mengisi `type_code` sesuai jenisnya tanpa pengecekan tambahan karena tipe sudah diketahui secara intrinsik
- Referensi variabel (`VarNode`) melakukan `lookup()`; jika tidak ditemukan melempar `undeclared_error`; jika ditemukan mengisi `type_code`, `tab_index`, dan `lev` dari `TabEntry`
- Operasi biner (`BinOpNode`) mengunjungi kedua operand lalu memanggil `resolve_binop_type()`; jika hasil UNKNOWN melempar `type_error`
- Operasi unari (`UnaryOpNode`) memvalidasi bahwa not hanya untuk BOOLEAN dan minus unari hanya untuk INTEGER atau REAL
- Assignment (`AssignNode`) memvalidasi kompatibilitas tipe antara target dan value menggunakan `is_assignment_compatible()`
- Statement if dan while memvalidasi bahwa kondisi bertipe BOOLEAN

- Statement repeat mengunjungi body terlebih dahulu sesuai semantiknya, baru kemudian kondisi divalidasi bertipe BOOLEAN
- Statement for memvalidasi bahwa variabel kontrol bertipe ordinal (INTEGER atau CHAR) dan ekspresi batas from serta to kompatibel dengan tipe variabel kontrol
- Statement case memvalidasi bahwa selector bertipe ordinal dan tipe setiap nilai case kompatibel dengan selector
- Pemanggilan prosedur dan fungsi melakukan lookup() untuk memvalidasi obj == PROCEDURE atau FUNCTION; return type fungsi diisi ke type_code node pemanggil
- Deklarasi variabel memanggil lookup_in_level() untuk cek duplikat, kemudian enter_tab() untuk mendaftarkan identifier, dan menginkrementasi btab.vsize
- Deklarasi prosedur dan fungsi mendaftarkan identifier ke tab, memanggil open_scope() untuk membuat blok baru, mengunjungi parameter dan body di dalam scope tersebut, lalu close_scope()

Fungsi Helper Type Checking

Terdapat tiga fungsi helper yang digunakan secara konsisten di seluruh visitor:

- is_compatible(t1, t2) mengembalikan true jika kedua tipe identik, atau salah satu adalah SUBRANGE dari INTEGER, atau keduanya STRING
- is_assignment_compatible(t1, t2) berlaku seperti is_compatible dengan tambahan satu aturan: INTEGER dapat di-assign ke variabel bertipe REAL sebagai bentuk implicit promotion, tetapi tidak berlaku sebaliknya
- resolve_binop_type(op, t1, t2) menentukan tipe hasil operasi biner berdasarkan operator dan tipe kedua operand sesuai tabel berikut:

Operator	Tipe Operand	Tipe Hasil
+ - *	Integer dan Integer	Integer
+ - *	Integer atau Real	Real
/	Integer atau Real	Real (selalu)
div mod	Integer dan Integer	Integer
and or	Boolean dan Boolean	Boolean
not	Boolean	Boolean
== < <> <= >=	Integer, Real, Char, atau String	Boolean
Selain kombinasi di atas	Kombinasi tidak valid	UNKNOWN, menghasilkan type_error

2.9

Penanganan Error Semantik

Semantic error ditangani dengan melempar exception `SemanticError`. Tiga jenis error yang didefinisikan: `type_error` untuk ketidaksesuaian tipe, `undeclared_error` untuk identifier yang belum dideklarasikan, dan `redeclared_error` untuk deklarasi ulang di scope yang sama. Program menghentikan analisis dan mencetak pesan error informatif ke `stderr` dengan exit code 1.

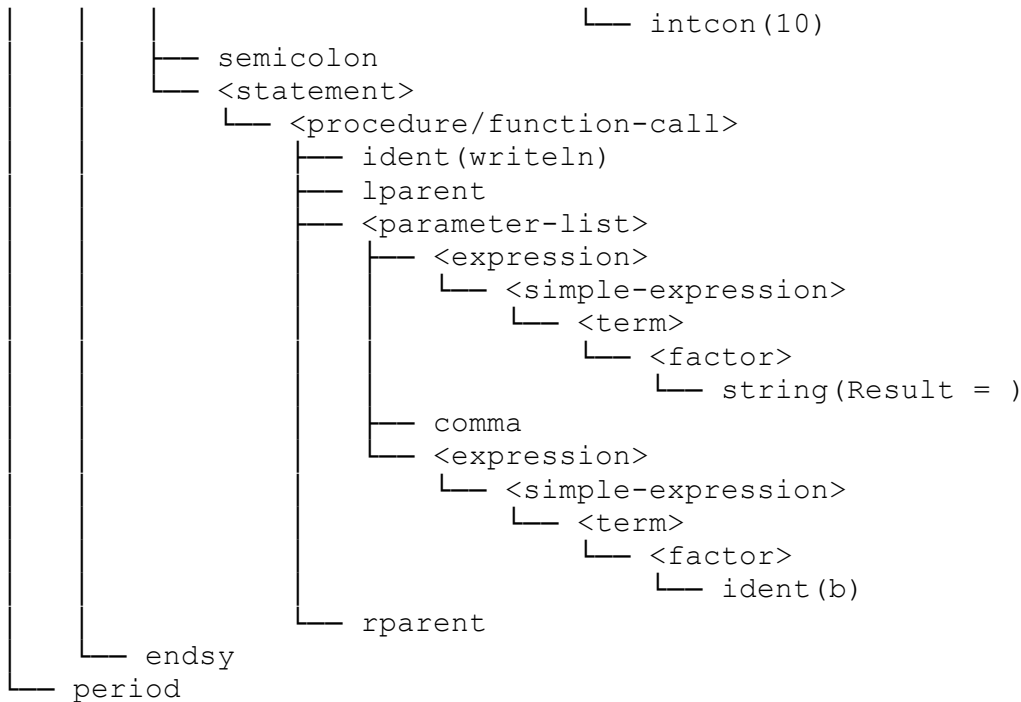
Bab III : Pengujian

3.1 Pengujian

3.1.1 Pengujian 1

Kode Program Input

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Hello)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (a)
│       │   ├── comma
│       │   └── ident (b)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   └── <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └── ident (a)
│   │   │       ├── becomes
│   │   │       └── <expression>
│   │   │           └── <simple-expression>
│   │   │               └── <term>
│   │   │                   └── <factor>
│   │   │                       └── intcon (5)
│   │   ├── semicolon
│   │   └── <statement>
│   │       └── <assignment-statement>
│   │           ├── <variable>
│   │           │   └── ident (b)
│   │           ├── becomes
│   │           └── <expression>
│   │               └── <simple-expression>
│   │                   ├── <term>
│   │                   │   ├── <factor>
│   │                   │   └── ident (a)
│   │                   ├── <additive-operator>
│   │                   │   └── plus
│   │                   └── <term>
│   │                       └── <factor>
```



Output Program

=== DECORATED AST ===

```

ProgramNode(name: 'Hello') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'a', 'b') [type:void, lev:0]
│   ├── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode(btab_index: 1) [type:void, lev:1]
│       ├── [stmt] AssignNode [type:void, lev:1]
│       │   ├── [target] VarNode(name: 'a') [tab_index:39, type:integer, lev:0]
│       │   └── [value] NumberNode(value: 5) [type:integer, lev:1]
│       ├── [stmt] AssignNode [type:void, lev:1]
│       │   ├── [target] VarNode(name: 'b') [tab_index:40, type:integer, lev:0]
│       │   └── [value] BinOpNode(op: '+') [type:integer, lev:1]
│       │       ├── [left] VarNode(name: 'a') [tab_index:39, type:integer, lev:0]
│       │       └── [right] NumberNode(value: 10) [type:integer, lev:1]
│       └── [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
│           ├── [arg] StringNode(value: "Result = ") [type:string, lev:1]
│           └── [arg] VarNode(name: 'b') [tab_index:40, type:integer, lev:0]
  
```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0

9 end	8 RESERVED	VOID	-1	1	0	0
10 for	9 RESERVED	VOID	-1	1	0	0
11 function	10 RESERVED	VOID	-1	1	0	0
12 if	11 RESERVED	VOID	-1	1	0	0
13 mod	12 RESERVED	VOID	-1	1	0	0
14 not	13 RESERVED	VOID	-1	1	0	0
15 of	14 RESERVED	VOID	-1	1	0	0
16 or	15 RESERVED	VOID	-1	1	0	0
17 procedure	16 RESERVED	VOID	-1	1	0	0
18 program	17 RESERVED	VOID	-1	1	0	0
19 record	18 RESERVED	VOID	-1	1	0	0
20 repeat	19 RESERVED	VOID	-1	1	0	0
21 integer	20 TYPE	INTEGER	-1	1	0	1
22 real	21 TYPE	REAL	-1	1	0	1
23 boolean	22 TYPE	BOOLEAN	-1	1	0	1
24 char	23 TYPE	CHAR	-1	1	0	1
25 string	24 TYPE	STRING	-1	1	0	1
26 then	25 RESERVED	VOID	-1	1	0	0
27 to	26 RESERVED	VOID	-1	1	0	0
28 type	27 RESERVED	VOID	-1	1	0	0
29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 Hello	37 PROGRAM	VOID	-1	1	0	0
39 a	38 VARIABLE	INTEGER	-1	1	0	0
40 b	39 VARIABLE	INTEGER	-1	1	0	1

BTAB

idx	last	lpar	psze	vsze
0	40	-1	0	2
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Screenshot

=== DECORATED AST ===

```

ProgramNode(name: 'Hello') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'a', 'b') [type:void, lev:0]
│   ├── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode(btab_index: 1) [type:void, lev:1]
│       ├── [stmt] AssignNode [type:void, lev:1]
│       │   ├── [target] VarNode(name: 'a') [tab_index:39, type:integer, lev:0]
│       │   └── [value] NumberNode(value: 5) [type:integer, lev:1]
│       ├── [stmt] AssignNode [type:void, lev:1]
│       │   ├── [target] VarNode(name: 'b') [tab_index:40, type:integer, lev:0]
│       │   └── [value] BinOpNode(op: '+') [type:integer, lev:1]
│       │       ├── [left] VarNode(name: 'a') [tab_index:39, type:integer, lev:0]
│       │       └── [right] NumberNode(value: 10) [type:integer, lev:1]
│       └── [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
│           ├── [arg] StringNode(value: "Result = ") [type:string, lev:1]
│           └── [arg] VarNode(name: 'b') [tab_index:40, type:integer, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0

29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 Hello	37 PROGRAM	VOID	-1	1	0	0
39 a	38 VARIABLE	INTEGER	-1	1	0	0
40 b	39 VARIABLE	INTEGER	-1	1	0	1

BTAB

idx	last	lpar	psze	vsze
0	40	-1	0	2
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

3.1.2

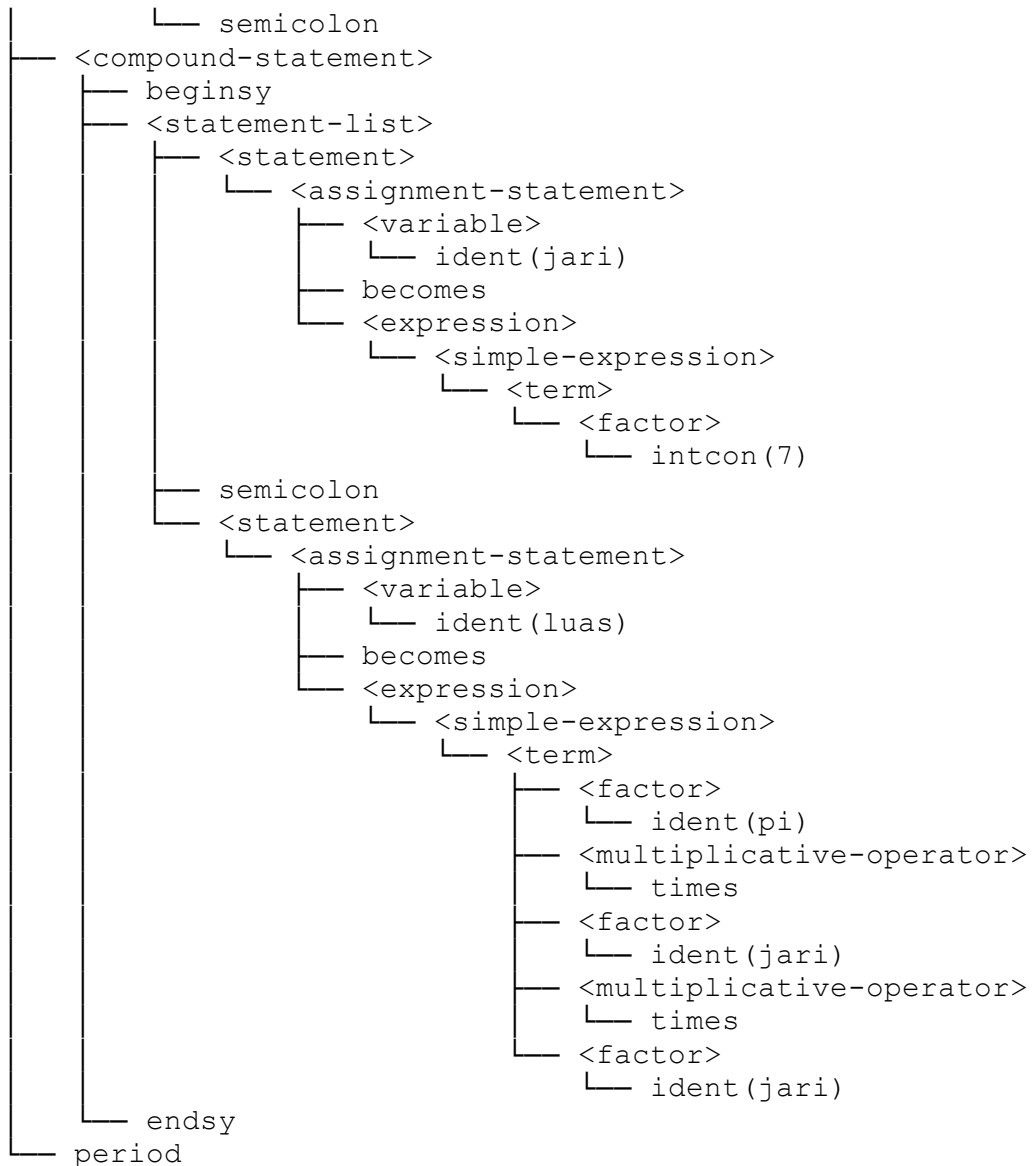
Pengujian 2

Kode Program Input

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(Aritmatika)
│   └── semicolon
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── constsy
│   │   ├── ident(pi)
│   │   ├── eql
│   │   ├── <constant>
│   │   │   └── realcon(3.14)
│   │   └── semicolon
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident(jari)
│       │   ├── comma
│       │   └── ident(luas)
│       ├── colon
│       ├── <type>
│       │   └── ident(real)

```



Output Program

=== DECORATED AST ===

```

ProgramNode(name: 'Aritmatika') [type:void, lev:0]
├── [decl] ConstDeclNode(name: 'pi') [type:void]
│   └── [value] RealNode(value: 3.14) [type:real, lev:0]
├── [decl] VarDeclNode(names: 'jari', 'luas') [type:void, lev:0]
│   └── [type] VarNode(name: 'real') [tab_index:22, type:real, lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] VarNode(name: 'jari') [tab_index:40, type:real, lev:0]
    │   └── [value] NumberNode(value: 7) [type:integer, lev:1]
    └── [stmt] AssignNode [type:void, lev:1]
        ├── [target] VarNode(name: 'luas') [tab_index:41, type:real, lev:0]
        └── [value] BinOpNode(op: '*') [type:real, lev:1]
  
```

```

└─ [left] BinOpNode(op: '*') [type:real, lev:1]
  └─ [left] VarNode(name: 'pi') [tab_index:39, type:real,
lev:0]
    └─ [right] VarNode(name: 'jari') [tab_index:40, type:real,
lev:0]
      └─ [right] VarNode(name: 'jari') [tab_index:40, type:real, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	Aritmatika	37	PROGRAM	VOID	-1	1	0	0
39	pi	38	CONSTANT	REAL	-1	1	0	0
40	jari	39	VARIABLE	REAL	-1	1	0	0
41	luas	40	VARIABLE	REAL	-1	1	0	1

BTAB

idx	last	lpar	psze	vsze
0	41	-1	0	2

	1	-1	-1	0	0	
 ATAB idx xtyp etyp eref low high elsz size						

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

etyp

Screenshot

=== DECORATED AST ===

```

ProgramNode(name: 'Aritmatika') [type:void, lev:0]
├── [decl] ConstDeclNode(name: 'pi') [type:void]
│   └── [value] RealNode(value: 3.14) [type:real, lev:0]
├── [decl] VarDeclNode(names: 'jari', 'luas') [type:void, lev:0]
│   └── [type] VarNode(name: 'real') [tab_index:22, type:real, lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] VarNode(name: 'jari') [tab_index:40, type:real, lev:0]
    │   └── [value] NumberNode(value: 7) [type:integer, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] VarNode(name: 'luas') [tab_index:41, type:real, lev:0]
    │   └── [value] BinOpNode(op: '*') [type:real, lev:1]
    │       ├── [left] BinOpNode(op: '*') [type:real, lev:1]
    │       │   ├── [left] VarNode(name: 'pi') [tab_index:39, type:real, lev:0]
    │       │   └── [right] VarNode(name: 'jari') [tab_index:40, type:real, lev:0]
    │       └── [right] VarNode(name: 'jari') [tab_index:40, type:real, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0

28 type	27 RESERVED	VOID	-1	1	0	0
29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 Aritmatika	37 PROGRAM	VOID	-1	1	0	0
39 pi	38 CONSTANT	REAL	-1	1	0	0
40 jari	39 VARIABLE	REAL	-1	1	0	0
41 luas	40 VARIABLE	REAL	-1	1	0	1

BTAB

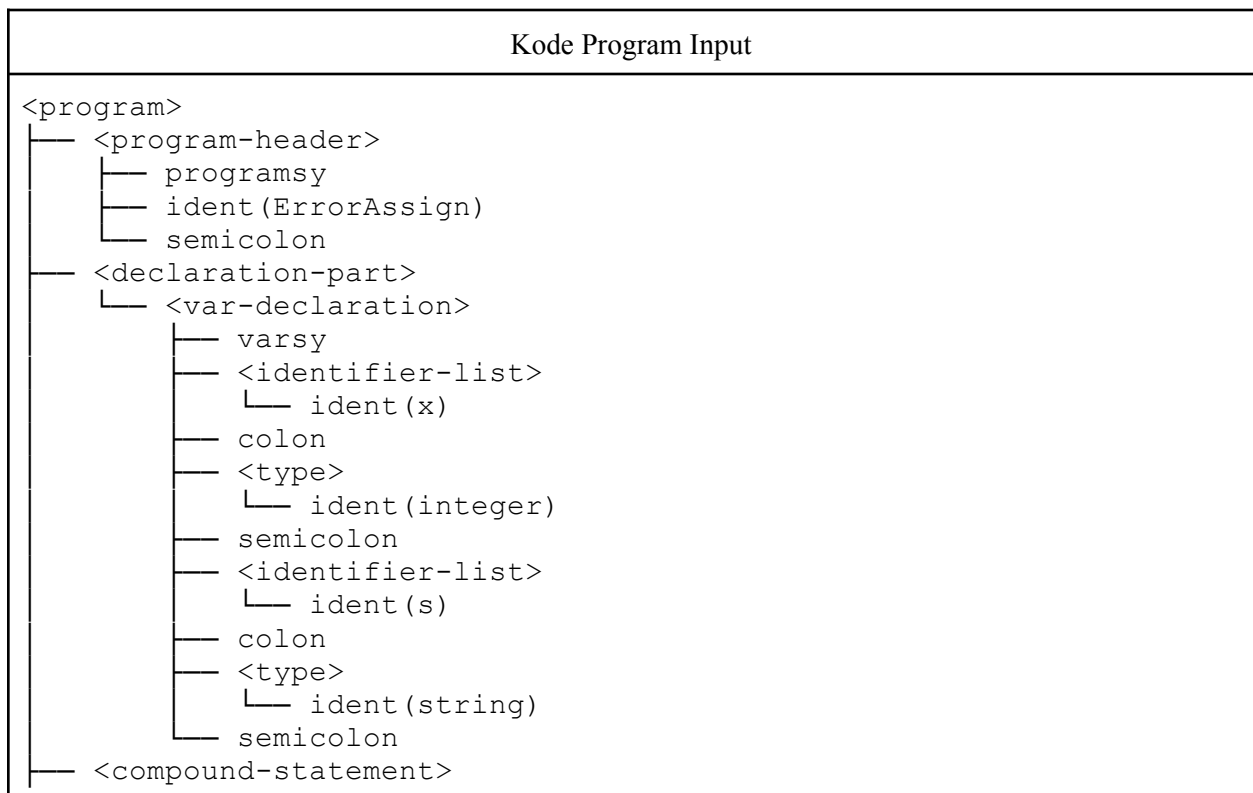
idx	last	lpar	psze	vsze
0	41	-1	0	2
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

3.1.3

Pengujian 3



```

├── beginsy
├── <statement-list>
│   ├── <statement>
│   │   ├── <assignment-statement>
│   │   │   ├── <variable>
│   │   │   │   └── ident(x)
│   │   │   ├── becomes
│   │   │   └── <expression>
│   │   │       ├── <simple-expression>
│   │   │       │   └── <term>
│   │   │       │       └── <factor>
│   │   │       │           └── intcon(10)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   ├── <assignment-statement>
│   │   │   │   ├── <variable>
│   │   │   │   │   └── ident(s)
│   │   │   │   ├── becomes
│   │   │   │   └── <expression>
│   │   │   │       ├── <simple-expression>
│   │   │   │       │   └── <term>
│   │   │   │       │       └── <factor>
│   │   │   │       │           └── string(halo)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   ├── <assignment-statement>
│   │   │   │   ├── <variable>
│   │   │   │   │   └── ident(x)
│   │   │   │   ├── becomes
│   │   │   │   └── <expression>
│   │   │   │       ├── <simple-expression>
│   │   │   │       │   └── <term>
│   │   │   │       │       └── <factor>
│   │   │   │       │           └── ident(s)
│   └── endsy
└── period

```

Output Program

Semantic Error: Assignment type mismatch: cannot assign String to Integer

Screenshot

```

naufal@LAPTOP-B15ASLP1:/mnt/c/Users/Naufal/OneDrive/Documents/SEMESTER_4/TBFO/Milest
one3/TBS-Tubes-IF2224-2026$ ./semantic test/semantic/input/input-tc3.txt
Semantic Error: Assignment type mismatch: cannot assign String to Integer

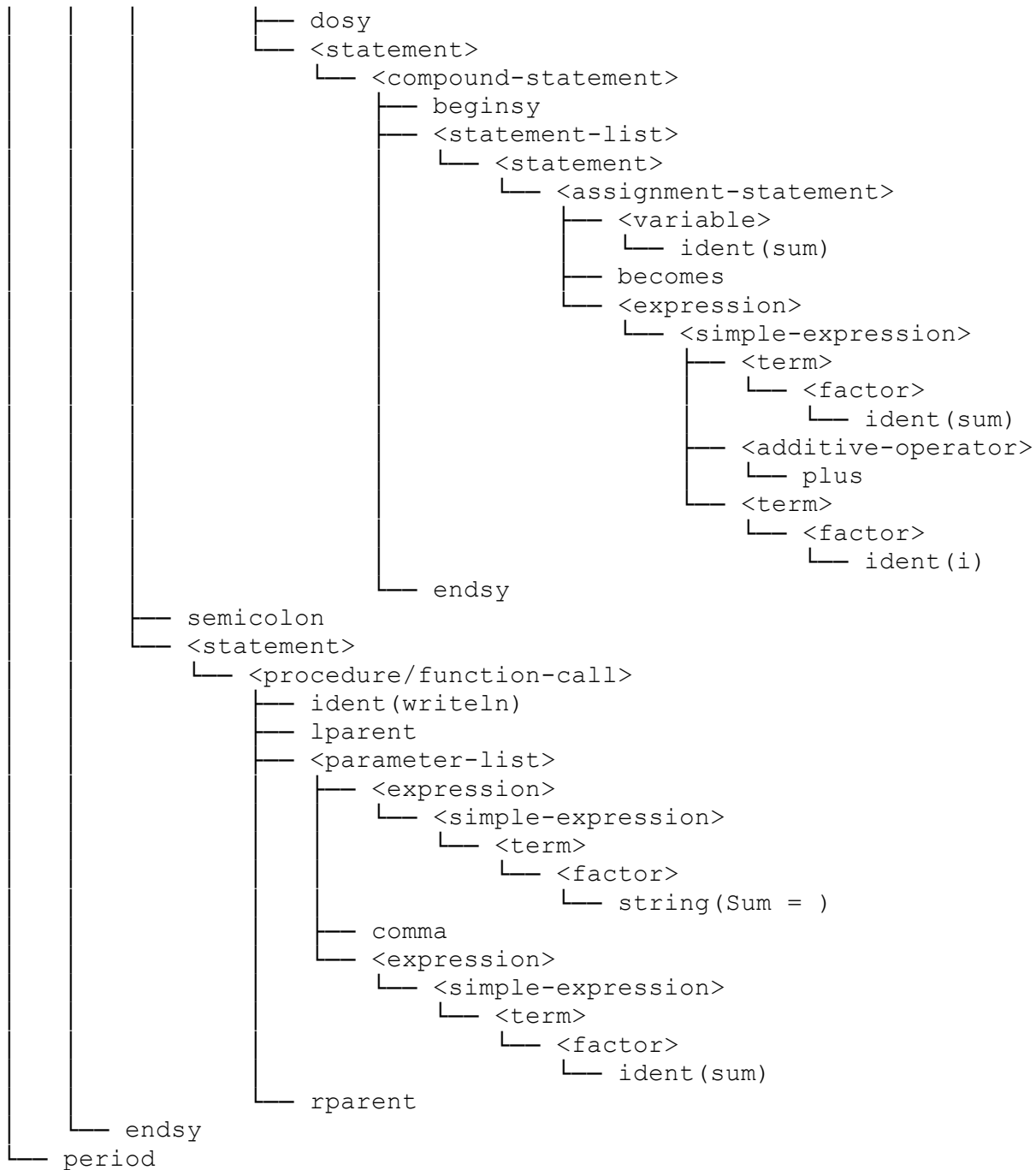
```


Kode Program Input

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(LoopTest)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident(i)
│       │   ├── comma
│       │   └── ident(sum)
│       ├── colon
│       ├── <type>
│       │   └── ident(integer)
│       └── semicolon
└── <compound-statement>
    ├── beginsy
    ├── <statement-list>
    │   ├── <statement>
    │   │   └── <assignment-statement>
    │   │       ├── <variable>
    │   │       │   └── ident(sum)
    │   │       ├── becomes
    │   │       └── <expression>
    │   │           └── <simple-expression>
    │   │               └── <term>
    │   │                   └── <factor>
    │   │                       └── intcon(0)
    │   ├── semicolon
    │   └── <statement>
    │       └── <for-statement>
    │           ├── forsy
    │           ├── ident(i)
    │           ├── becomes
    │           ├── <expression>
    │           │   └── <simple-expression>
    │           │       └── <term>
    │           │           └── <factor>
    │           │               └── intcon(1)
    │           ├── tosy
    │           ├── <expression>
    │           │   └── <simple-expression>
    │           │       └── <term>
    │           │           └── <factor>
    │           │               └── intcon(10)

```



Output Program

=== DECORATED AST ===

ProgramNode(name: 'LoopTest') [type:void, lev:0]

└─ [decl] VarDeclNode(names: 'i', 'sum') [type:void, lev:0]

└─ └─ [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]

└─ [body] BlockNode(btab_index: 1) [type:void, lev:1]

```

└─ [stmt] AssignNode [type:void, lev:1]
    └─ [target] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]
        └─ [value] NumberNode(value: 0) [type:integer, lev:1]
└─ [stmt] ForNode(var: 'i', dir: to) [type:void, lev:1]
    └─ [from] NumberNode(value: 1) [type:integer, lev:1]
        └─ [to] NumberNode(value: 10) [type:integer, lev:1]
            └─ [body] BlockNode [type:void, lev:1]
                └─ [stmt] AssignNode [type:void, lev:1]
                    └─ [target] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]
                        └─ [value] BinOpNode(op: '+') [type:integer, lev:1]
                            └─ [left] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]
                                └─ [right] VarNode(name: 'i') [tab_index:39, type:integer, lev:0]
└─ [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
    └─ [arg] StringNode(value: "Sum = ") [type:string, lev:1]
        └─ [arg] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0

37 read	36 PROCEDURE	VOID	-1	1	0	0
38 LoopTest	37 PROGRAM	VOID	-1	1	0	0
39 i	38 VARIABLE	INTEGER	-1	1	0	0
40 sum	39 VARIABLE	INTEGER	-1	1	0	1

BTAB

idx	last	lpar	psize	vsze
0	40	-1	0	2
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Screenshot

=== DECORATED AST ===

```

ProgramNode(name: 'LoopTest') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'i', 'sum') [type:void, lev:0]
│   ├── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode(btab_index: 1) [type:void, lev:1]
│       ├── [stmt] AssignNode [type:void, lev:1]
│       │   ├── [target] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]
│       │   └── [value] NumberNode(value: 0) [type:integer, lev:1]
│       ├── [stmt] ForNode(var: 'i', dir: to) [type:void, lev:1]
│       │   ├── [from] NumberNode(value: 1) [type:integer, lev:1]
│       │   ├── [to] NumberNode(value: 10) [type:integer, lev:1]
│       │   └── [body] BlockNode [type:void, lev:1]
│       │       └── [stmt] AssignNode [type:void, lev:1]
│       │           ├── [target] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]
│       │           └── [value] BinOpNode(op: '+') [type:integer, lev:1]
│       │               ├── [left] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]
│       │               └── [right] VarNode(name: 'i') [tab_index:39, type:integer, lev:0]
│       └── [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
│           ├── [arg] StringNode(value: "Sum = ") [type:string, lev:1]
│           └── [arg] VarNode(name: 'sum') [tab_index:40, type:integer, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1

25 string	24 TYPE	STRING	-1	1	0	1
26 then	25 RESERVED	VOID	-1	1	0	0
27 to	26 RESERVED	VOID	-1	1	0	0
28 type	27 RESERVED	VOID	-1	1	0	0
29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 LoopTest	37 PROGRAM	VOID	-1	1	0	0
39 i	38 VARIABLE	INTEGER	-1	1	0	0
40 sum	39 VARIABLE	INTEGER	-1	1	0	1

BTAB

idx	last	lpar	psze	vsze
0	40	-1	0	2
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

3.1.5

Pengujian 5

Kode Program Input

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(Undeclared)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident(a)
│       ├── colon
│       ├── <type>
│       │   └── ident(integer)
│       └── semicolon
└── <compound-statement>
    ├── beginsy
    └── <statement-list>
        └── <statement>

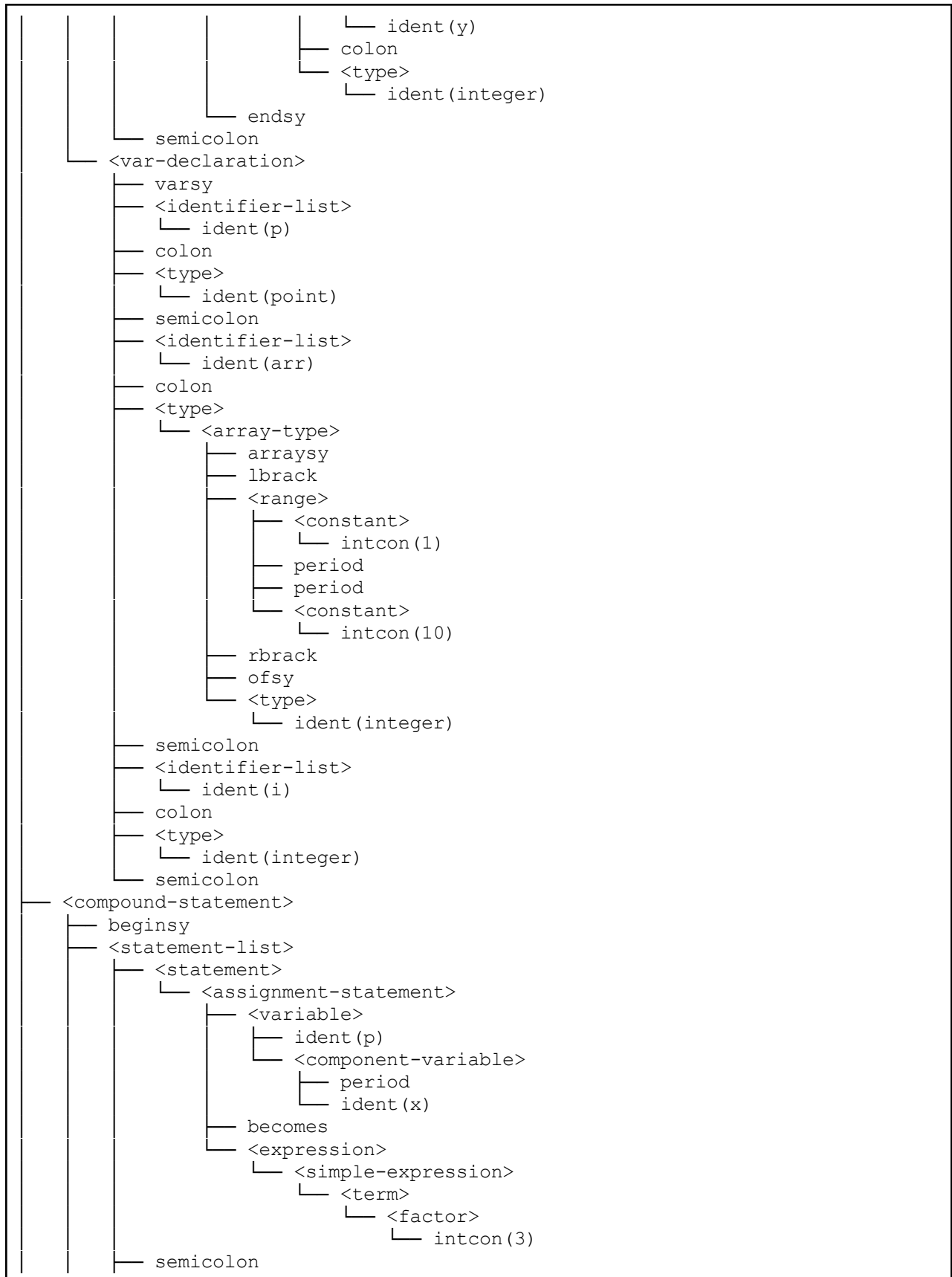
```

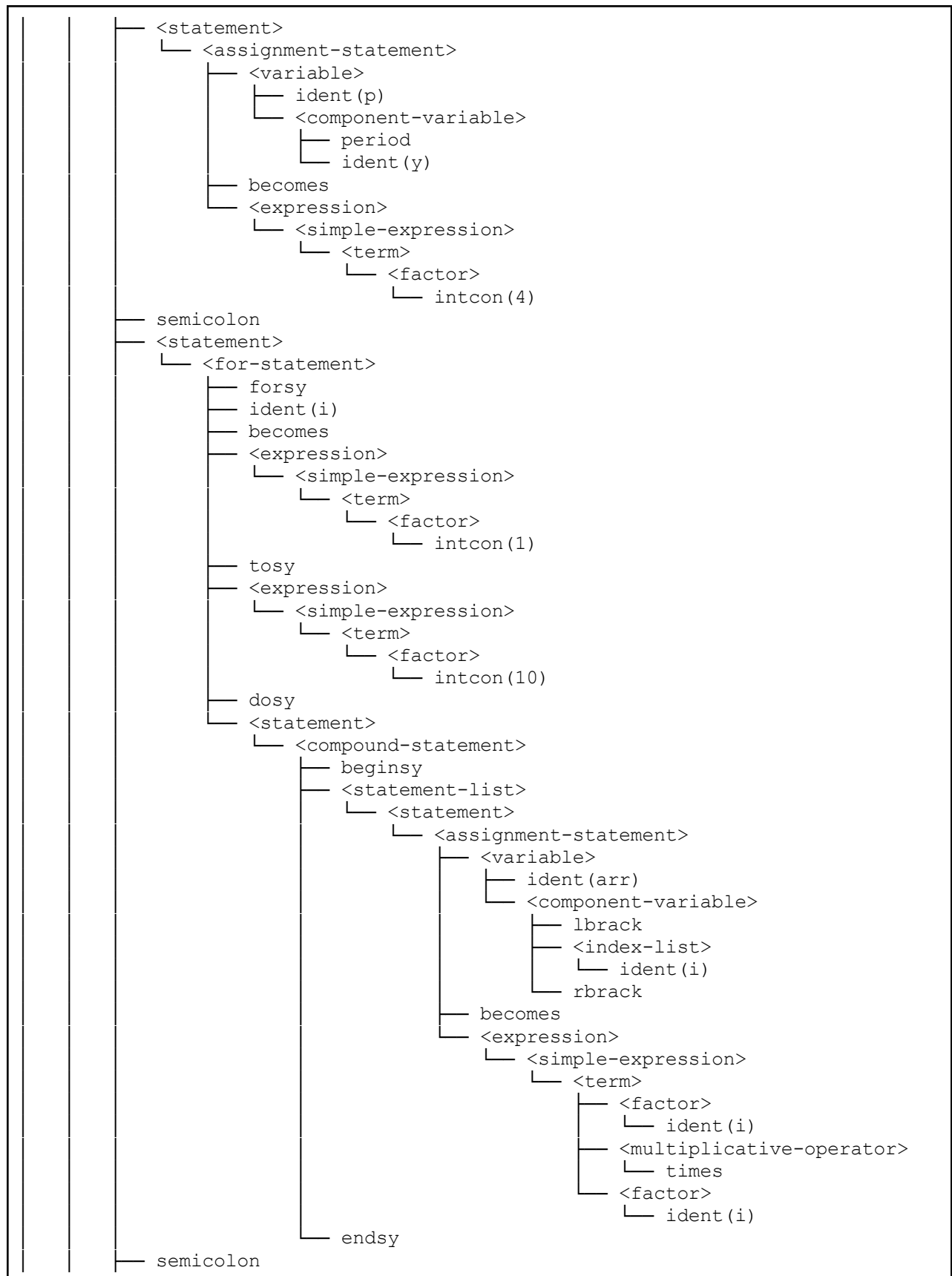
<pre> └─ <assignment-statement> └─ <variable> └─ ident(a) └─ becomes └─ <expression> └─ <simple-expression> └─ <term> └─ <factor> └─ ident(b) └─ <additive-operator> └─ plus └─ <term> └─ <factor> └─ intcon(1) └─ endsy └─ period </pre>
Output Program
Semantic Error: Undeclared identifier: 'b'
Screenshot
<pre> naufal@LAPTOP-B15ASLP1:/mnt/c/Users/Naufal/OneDrive/Documents/SEMESTER_4/TBFO/Milest one3/TBS-Tubes-IF2224-2026\$./semantic test/semantic/input/input-tc5.txt Semantic Error: Undeclared identifier: 'b' </pre>

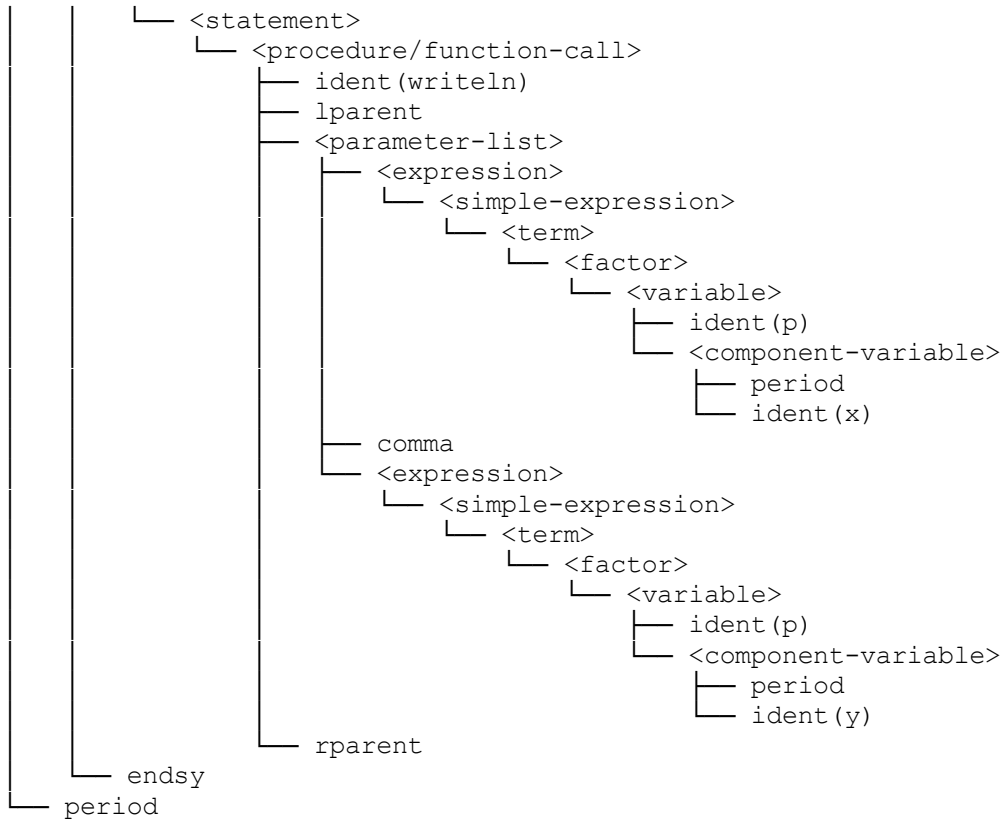
3.1.6

Pengujian 6

Kode Program Input
<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(Geometri) │ └─ semicolon ├─ <declaration-part> │ ├── <type-declaration> │ │ ├── typesy │ │ ├── ident(point) │ │ ├── eql │ │ └─ <type> │ │ └─ <record-type> │ │ ├── recordsy │ │ └─ <field-list> │ │ └─ <field-part> │ │ └─ <identifier-list> │ │ ├── ident(x) │ │ └─ comma </pre>







Output Program

```

=== DECORATED AST ===
ProgramNode(name: 'Geometri') [type:void, lev:0]
├── [decl] TypeDeclNode(name: 'point') [type:void]
│   ├── [type] RecordTypeNode [tab_index:1, type:record, lev:0]
│   │   ├── [field] VarDeclNode(names: 'x', 'y') [type:void, lev:1]
│   │   │   ├── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   │   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] VarDeclNode(names: 'p') [type:void, lev:0]
│   ├── [type] VarNode(name: 'point') [tab_index:41, type:record, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] VarDeclNode(names: 'arr') [type:void, lev:0]
│   ├── [type] ArrayTypeNode [tab_index:0, type:array, lev:0]
│   │   ├── [index] SubrangeNode [type:integer, lev:0]
│   │   │   ├── [low] NumberNode(value: 1) [type:integer, lev:0]
│   │   │   └── [high] NumberNode(value: 10) [type:integer, lev:0]
│   │   └── [elem] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] VarDeclNode(names: 'i') [type:void, lev:0]
│   ├── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
└── [body] BlockNode(btab_index: 2) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] RecordAccessNode(field: 'x') [tab_index:39, type:integer, lev:1]
    │   │   ├── [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]
    │   │   └── [value] NumberNode(value: 3) [type:integer, lev:1]
    │   └── [value] NumberNode(value: 3) [type:integer, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] RecordAccessNode(field: 'y') [tab_index:40, type:integer, lev:1]
    │   │   ├── [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]
    │   │   └── [value] NumberNode(value: 4) [type:integer, lev:1]
    │   └── [value] NumberNode(value: 4) [type:integer, lev:1]
    └── [stmt] ForNode(var: 'i', dir: to) [type:void, lev:1]
  
```

```

└─ [from] NumberNode(value: 1) [type:integer, lev:1]
└─ [to] NumberNode(value: 10) [type:integer, lev:1]
└─ [body] BlockNode [type:void, lev:1]
    └─ [stmt] AssignNode [type:void, lev:1]
        └─ [target] ArrayAccessNode [type:integer, lev:1]
            └─ [array] VarNode(name: 'arr') [tab_index:43, type:array, lev:0]
            └─ [index] VarNode(name: 'i') [tab_index:44, type:integer, lev:0]
            └─ [value] BinOpNode(op: '*') [type:integer, lev:1]
                └─ [left] VarNode(name: 'i') [tab_index:44, type:integer, lev:0]
                └─ [right] VarNode(name: 'i') [tab_index:44, type:integer, lev:0]
└─ [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
    └─ [arg] RecordAccessNode(field: 'x') [tab_index:39, type:integer, lev:1]
        └─ [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]
    └─ [arg] RecordAccessNode(field: 'y') [tab_index:40, type:integer, lev:1]
        └─ [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	Geometri	37	PROGRAM	VOID	-1	1	0	0
39	x	-1	VARIABLE	INTEGER	-1	1	1	0
40	y	39	VARIABLE	INTEGER	-1	1	1	1
41	point	38	TYPE	RECORD	1	1	0	0
42	p	41	VARIABLE	RECORD	1	1	0	0

43	arr	42	VARIABLE	ARRAY	0	1	0	1
44	i	43	VARIABLE	INTEGER	-1	1	0	2

BTAB

idx	last	lpar	psze	vsze
0	44	-1	0	3
1	40	-1	0	2
2	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
0	INTEGER	INTEGER	21	1	10	1	10

Screenshot

=== DECORATED AST ===

```

ProgramNode(name: 'Geometri') [type:void, lev:0]
├── [decl] TypeDeclNode(name: 'point') [type:void]
│   └── [type] RecordTypeNode [tab_index:1, type:record, lev:0]
│       └── [field] VarDeclNode(names: 'x', 'y') [type:void, lev:1]
│           └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] VarDeclNode(names: 'p') [type:void, lev:0]
│   └── [type] VarNode(name: 'point') [tab_index:41, type:record, lev:0]
├── [decl] VarDeclNode(names: 'arr') [type:void, lev:0]
│   └── [type] ArrayTypeNode [tab_index:0, type:array, lev:0]
│       ├── [index] SubrangeNode [type:integer, lev:0]
│       │   ├── [low] NumberNode(value: 1) [type:integer, lev:0]
│       │   └── [high] NumberNode(value: 10) [type:integer, lev:0]
│       └── [elem] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] VarDeclNode(names: 'i') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
└── [body] BlockNode(btab_index: 2) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] RecordAccessNode(field: 'x') [tab_index:39, type:integer, lev:1]
    │   │   └── [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]
    │   └── [value] NumberNode(value: 3) [type:integer, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] RecordAccessNode(field: 'y') [tab_index:40, type:integer, lev:1]
    │   │   └── [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]
    │   └── [value] NumberNode(value: 4) [type:integer, lev:1]
    ├── [stmt] ForNode(var: 'i', dir: to) [type:void, lev:1]
    │   ├── [from] NumberNode(value: 1) [type:integer, lev:1]
    │   ├── [to] NumberNode(value: 10) [type:integer, lev:1]
    │   └── [body] BlockNode [type:void, lev:1]
    │       └── [stmt] AssignNode [type:void, lev:1]
    │           ├── [target] ArrayAccessNode [type:integer, lev:1]
    │           │   ├── [array] VarNode(name: 'arr') [tab_index:43, type:array, lev:0]
    │           │   ├── [index] VarNode(name: 'i') [tab_index:44, type:integer, lev:0]
    │           │   └── [value] BinOpNode(op: '*') [type:integer, lev:1]
    │           │       ├── [left] VarNode(name: 'i') [tab_index:44, type:integer, lev:0]
    │           │       └── [right] VarNode(name: 'i') [tab_index:44, type:integer, lev:0]
    └── [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
        ├── [arg] RecordAccessNode(field: 'x') [tab_index:39, type:integer, lev:1]
        │   └── [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]
        └── [arg] RecordAccessNode(field: 'y') [tab_index:40, type:integer, lev:1]
            └── [record] VarNode(name: 'p') [tab_index:42, type:record, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0

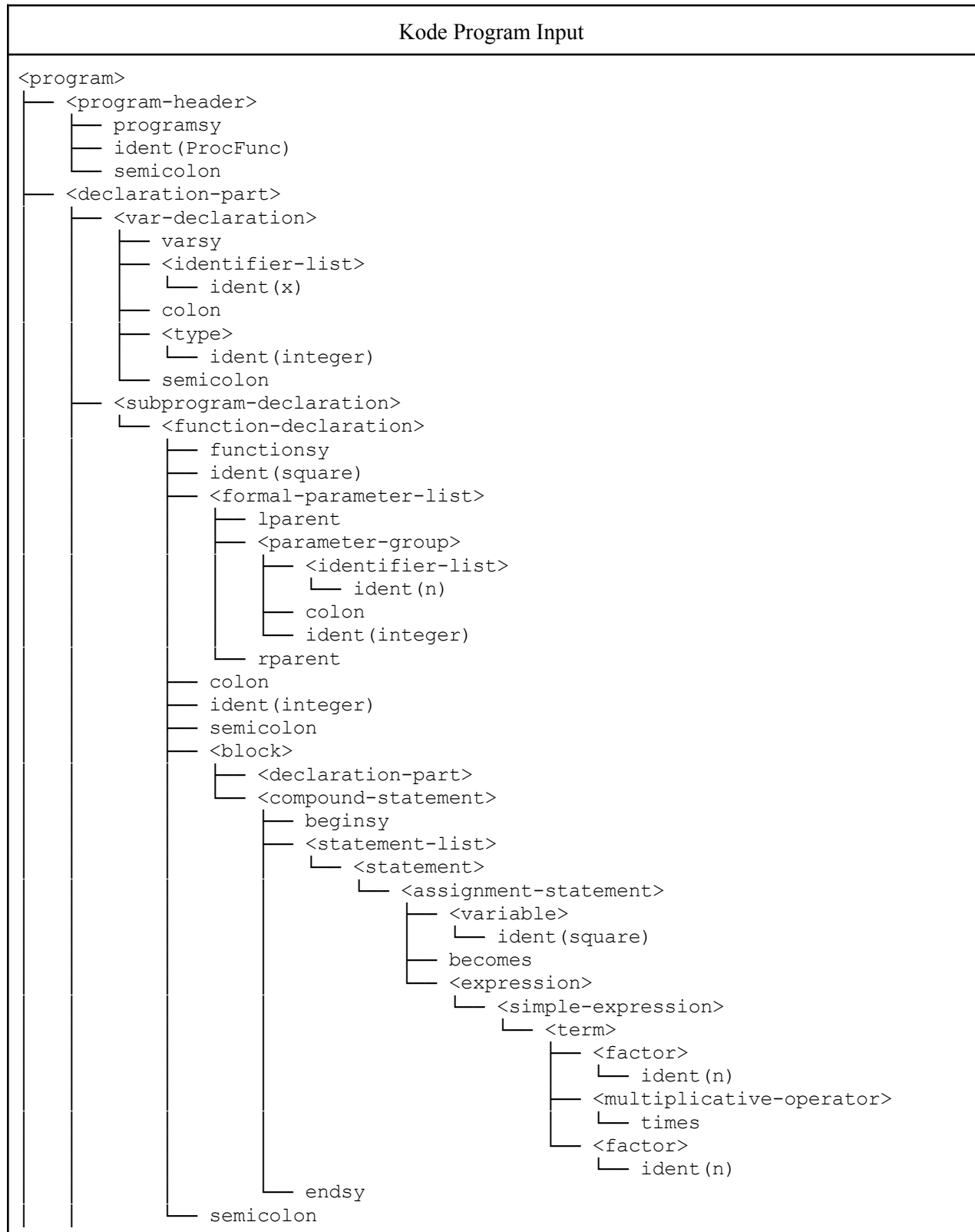
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	Geometri	37	PROGRAM	VOID	-1	1	0	0
39	x	-1	VARIABLE	INTEGER	-1	1	1	0
40	y	39	VARIABLE	INTEGER	-1	1	1	1
41	point	38	TYPE	RECORD	1	1	0	0
42	p	41	VARIABLE	RECORD	1	1	0	0
43	arr	42	VARIABLE	ARRAY	0	1	0	1
44	i	43	VARIABLE	INTEGER	-1	1	0	2

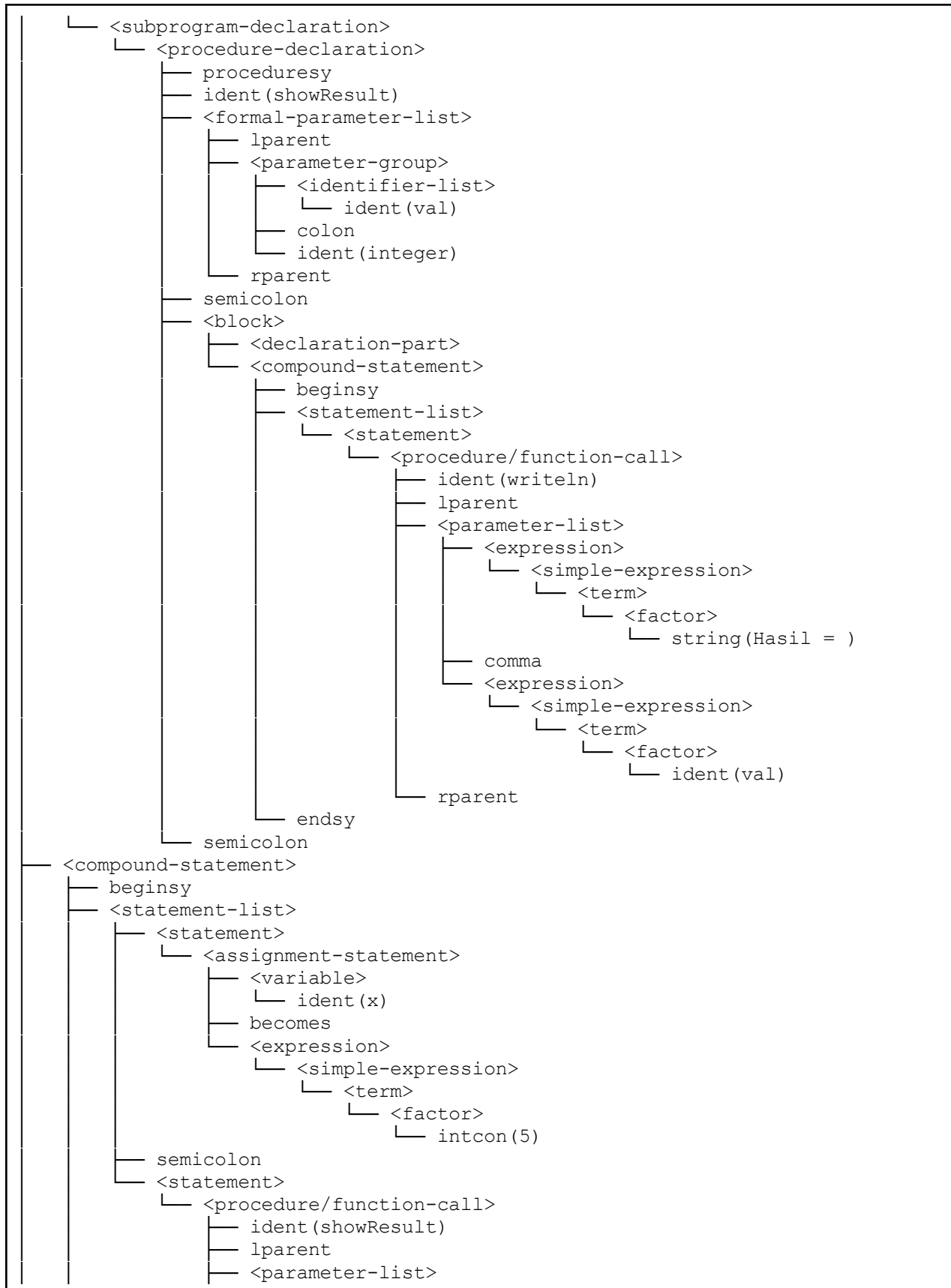
BTAB

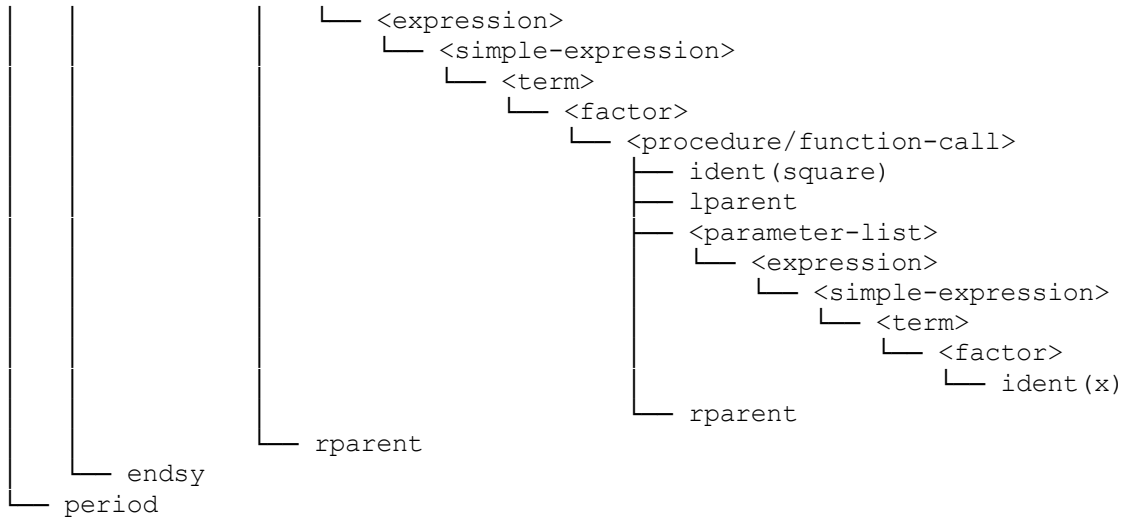
idx	last	lpar	psze	vsze
0	44	-1	0	3
1	40	-1	0	2
2	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
0	INTEGER	INTEGER	21	1	10	1	10







Output Program

=== DECORATED AST ===

```

ProgramNode(name: 'ProcFunc') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] FuncDeclNode(name: 'square') [type:void]
│   ├── [param] ParamGroupNode(var: false, names: 'n') [type:void]
│   │   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   ├── [returns] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode [type:void, lev:1]
│       └── [stmt] AssignNode [type:void, lev:1]
│           ├── [target] VarNode(name: 'square') [tab_index:40, type:integer, lev:0]
│           └── [value] BinOpNode(op: '*') [type:integer, lev:1]
│               ├── [left] VarNode(name: 'n') [tab_index:41, type:integer, lev:1]
│               └── [right] VarNode(name: 'n') [tab_index:41, type:integer, lev:1]
├── [decl] ProcDeclNode(name: 'showResult') [type:void]
│   ├── [param] ParamGroupNode(var: false, names: 'val') [type:void]
│   │   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode [type:void, lev:1]
│       ├── [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
│       │   ├── [arg] StringNode(value: "Hasil = ") [type:string, lev:1]
│       │   └── [arg] VarNode(name: 'val') [tab_index:43, type:integer, lev:1]
│       └── [body] BlockNode(btab_index: 3) [type:void, lev:1]
│           ├── [stmt] AssignNode [type:void, lev:1]
│           │   ├── [target] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]
│           │   └── [value] NumberNode(value: 5) [type:integer, lev:1]
│           └── [stmt] ProcCallNode(name: 'showResult') [tab_index:42, type:void, lev:1]
│               ├── [arg] FuncCallExprNode(name: 'square') [tab_index:40, type:integer, lev:1]
│               │   └── [arg] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0

4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	ProcFunc	37	PROGRAM	VOID	-1	1	0	0
39	x	38	VARIABLE	INTEGER	-1	1	0	0
40	square	39	FUNCTION	INTEGER	1	1	0	0
41	n	-1	VARIABLE	INTEGER	-1	1	1	0
42	showResult	40	PROCEDURE	VOID	2	1	0	0
43	val	-1	VARIABLE	INTEGER	-1	1	1	0

BTAB

idx	last	lpar	psze	vsze
0	42	-1	0	1
1	41	41	1	0
2	43	43	1	0
3	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Screenshot

=== DECORATED AST ===

```

ProgramNode(name: 'ProcFunc') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] FuncDeclNode(name: 'square') [type:void]
│   ├── [param] ParamGroupNode(var: false, names: 'n') [type:void]
│   │   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   ├── [returns] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode [type:void, lev:1]
│       └── [stmt] AssignNode [type:void, lev:1]
│           ├── [target] VarNode(name: 'square') [tab_index:40, type:integer, lev:0]
│           └── [value] BinOpNode(op: '*') [type:integer, lev:1]
│               ├── [left] VarNode(name: 'n') [tab_index:41, type:integer, lev:1]
│               └── [right] VarNode(name: 'n') [tab_index:41, type:integer, lev:1]
├── [decl] ProcDeclNode(name: 'showResult') [type:void]
│   ├── [param] ParamGroupNode(var: false, names: 'val') [type:void]
│   │   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode [type:void, lev:1]
│       ├── [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void, lev:1]
│       │   ├── [arg] StringNode(value: "Hasil = ") [type:string, lev:1]
│       │   └── [arg] VarNode(name: 'val') [tab_index:43, type:integer, lev:1]
│       └── [body] BlockNode(btab_index: 3) [type:void, lev:1]
│           ├── [stmt] AssignNode [type:void, lev:1]
│           │   ├── [target] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]
│           │   └── [value] NumberNode(value: 5) [type:integer, lev:1]
│           └── [stmt] ProcCallNode(name: 'showResult') [tab_index:42, type:void, lev:1]
│               ├── [arg] FuncCallExprNode(name: 'square') [tab_index:40, type:integer, lev:1]
│               └── [arg] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0

17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	ProcFunc	37	PROGRAM	VOID	-1	1	0	0
39	x	38	VARIABLE	INTEGER	-1	1	0	0
40	square	39	FUNCTION	INTEGER	1	1	0	0
41	n	-1	VARIABLE	INTEGER	-1	1	1	0
42	showResult	40	PROCEDURE	VOID	2	1	0	0
43	val	-1	VARIABLE	INTEGER	-1	1	1	0

BTAB

idx	last	lpar	psze	vsze
0	42	-1	0	1
1	41	41	1	0
2	43	43	1	0
3	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

3.1.8

Pengujian 8

Kode Program Input

```

<program>
├─ <program-header>
│   └─ programsy

```

```

├── ident (Redecl)
├── semicolon
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   └── ident (x)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (integer)
│   │   ├── semicolon
│   │   ├── <identifier-list>
│   │   │   └── ident (x)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (real)
│   │   └── semicolon
│   ├── <compound-statement>
│   │   ├── beginsy
│   │   ├── <statement-list>
│   │   │   ├── <statement>
│   │   │   │   ├── <assignment-statement>
│   │   │   │   │   ├── <variable>
│   │   │   │   │   │   └── ident (x)
│   │   │   │   │   ├── becomes
│   │   │   │   │   └── <expression>
│   │   │   │   │       ├── <simple-expression>
│   │   │   │   │       │   ├── <term>
│   │   │   │   │       │   │   └── <factor>
│   │   │   │   │       │   │       └── intcon (0)
│   │   │   │   └── semicolon
│   │   │   └── <statement>
│   │   └── endsy
│   └── period

```

Output Program

Semantic Error: Identifier already declared in this scope: 'x'

Screenshot

```

naufal@LAPTOP-B15ASLP1:/mnt/c/Users/Naufal/OneDrive/Documents/SEMESTER_4/TBFO/Milest
one3/TBS-Tubes-IF2224-2026$ ./semantic test/semantic/input/input-tc8.txt
Semantic Error: Identifier already declared in this scope: 'x'

```

3.2 Analisis Pengujian

3.2.1 Pengujian 1

Pengujian pertama menguji alur dasar semantik pada program yang mendeklarasikan dua variabel integer (a dan b) lalu melakukan assignment serta operasi penjumlahan. Hasil yang diharapkan adalah program lolos analisis semantik tanpa error, dan keluaran ini sesuai harapan.

Dari Decorated AST terlihat bahwa `SemanticVisitor` berhasil melakukan lookup terhadap kedua variabel, yaitu a memperoleh `tab_index:39` dan b memperoleh `tab_index:40`, keduanya dianotasi `type:integer`. Pada statement `b := a + 10`, node `BinOpNode(op:'+')` dianotasi `type:integer` karena kedua operand bertipe integer. Hal ini sesuai aturan type checking bahwa operasi aritmatika menghasilkan integer jika kedua operand integer. Assignment ke b valid karena tipe kiri dan kanan sama-sama integer. Pada pemanggilan `writeln`, argumen bertipe `string` dan `integer` diterima karena `writeln` merupakan predefined procedure yang dirancang menerima argumen campuran.

Pada Symbol Table, indeks 0–37 terisi reserved words dan predefined identifiers, identifier pengguna dimulai dari indeks 38 (`Hello` sebagai PROGRAM), diikuti a dan b sebagai VARIABLE INTEGER dengan `adr` 0 dan 1 yang menunjukkan alokasi alamat berurutan untuk variabel lokal. Pada `btabs`, blok global (`idx` 0) mencatat `vsz:2` yang konsisten dengan dua variabel integer berukuran satu satuan. Pengujian ini memvalidasi bahwa mekanisme dasar lookup, anotasi tipe, dan alokasi alamat berjalan benar.

3.2.2 Pengujian 2

Pengujian kedua menguji penanganan deklarasi konstanta (`pi = 3.14`) dan variabel bertipe real, sekaligus menguji operasi perkalian berantai. Program diharapkan lolos tanpa error, dan hasilnya sesuai.

Aspek penting yang diuji di sini adalah promosi tipe implisit. Pada statement `jari := 7`, variabel `jari` bertipe real sedangkan literal 7 bertipe integer. Decorated AST menunjukkan `NumberNode(value:7)` tetap dianotasi `type:integer`, namun assignment tetap diterima. Hal ini membuktikan fungsi `is_assignment_compatible` bekerja benar, karena nilai integer boleh di-assign ke variabel real (integer dapat dipromosikan ke real). Sebaliknya, kasus kebalikannya akan ditolak.

Pada statement `luas := pi * jari * jari`, struktur AST membentuk `BinOpNode('*')` bersarang dengan asosiativitas kiri, sesuai cara `build_term()` menangani operator perkalian. Setiap `BinOpNode` dianotasi `type:real` karena salah satu operand (`pi`) bertipe real yang konsisten dengan aturan bahwa operasi aritmatika menghasilkan real bila ada operand real. Pada Symbol Table, `pi` tercatat sebagai

CONSTANT REAL dan jari/luas sebagai VARIABLE REAL. Pengujian ini memvalidasi perbedaan kelas objek CONSTANT vs VARIABLE serta benarnya aturan `resolve_binop_type` untuk tipe campuran.

3.2.3 Pengujian 3

Pengujian ketiga adalah testcase negatif pertama, yang sengaja dirancang menghasilkan error. Program mendeklarasikan `x: integer` dan `s: string`, lalu pada statement terakhir melakukan `x := s`. Keluaran yang diharapkan adalah penghentian analisis dengan pesan error, dan hasilnya sesuai, yaitu “Semantic Error: Assignment type mismatch: cannot assign String to Integer”.

Dua statement sebelumnya (`x := 10` dan `s := 'halo'`) valid karena tipe sisi kiri dan kanan cocok. Error baru terdeteksi pada statement ketiga ketika `SemanticVisitor` mengunjungi `AssignNode`, dimana setelah lookup, target bertipe `integer` dan nilai bertipe `string`. Fungsi `is_assignment_compatible` mengembalikan false karena string bukan integer, sehingga visitor melempar exception `SemanticError` jenis `type_error`. Program berhenti pada error pertama dan mencetak pesan ke `stderr` dengan exit code 1. Pengujian ini membuktikan bahwa analyzer tidak hanya memvalidasi program benar, tetapi juga berhasil mendeteksi pelanggaran assignment compatibility.

3.2.4 Pengujian 4

Pengujian keempat menguji konstruksi perulangan `for` beserta variabel kontrolnya. Program menghitung akumulasi `sum` dari 1 sampai 10. Program diharapkan lolos tanpa error, dan hasilnya sesuai.

Aspek yang divalidasi adalah aturan bahwa variabel kontrol `for` harus bertipe ordinal. Decorated AST menunjukkan `ForNode(var:'i', dir:to)` dengan variabel `i` bertipe `integer` yang memenuhi syarat ordinal. Batas `from` dan `to` keduanya `NumberNode` bertipe `integer`, sehingga valid. Di dalam badan loop, statement `sum := sum + i` dianotasi dengan benar, dimana `BinOpNode('+')` bertipe `integer`, dan assignment ke `sum` (`integer`) valid.

Pada Symbol Table dapat dilihat variabel `i` dan `sum` tetap berada di blok global (`lev:0`, `btabs:0`) yang mana badan `for` tidak membentuk scope baru, melainkan hanya `BlockNode` statement biasa. Ini terlihat dari `btabs` yang hanya berisi dua entri (global dan satu blok kosong), serta `vsz:2` pada blok global. Pengujian ini memvalidasi penanganan struktur kontrol perulangan dan benarnya pengecekan tipe variabel kontrol.

3.2.5 Pengujian 5

Pengujian kelima adalah testcase negatif kedua, menguji deteksi penggunaan identifier yang belum dideklarasikan. Program mendeklarasikan variabel `a`, tetapi pada statement `a`

`:= b + 1` menggunakan identifier `b` yang tidak pernah dideklarasikan. Keluaran yang diharapkan adalah error, dan hasilnya sesuai, yaitu “Semantic Error: Undeclared identifier: 'b'”.

Saat `SemanticVisitor` mengunjungi `VarNode` untuk `b`, ia memanggil `lookup("b")` yang menelusuri scope dari level terdalam ke terluar. Karena `b` tidak ditemukan di level manapun, `lookup` gagal dan visitor melempar `SemanticError` jenis `undeclared_error`. Analisis berhenti sebelum sempat menyelesaikan anotasi statement tersebut.

Pengujian ini melengkapi cakupan deteksi error, yaitu bersama Pengujian 3 (type error), keduanya menunjukkan dua kategori `SemanticError` yang berbeda berhasil ditangani. Pengujian ini juga secara tidak langsung memvalidasi benarnya mekanisme `lookup` dan ia mengembalikan kegagalan ketika identifier memang tidak ada, bukan keliru mengembalikan entri lain.

3.2.6 Pengujian 6

Pengujian keenam adalah testcase paling kompleks, menguji tipe data komposit, yaitu deklarasi tipe `record` (`point` dengan field `x`, `y`) dan tipe `array` (`arr: array[1..10] of integer`). Program diharapkan lolos tanpa error, dan hasilnya sesuai.

Pada penanganan `record`, deklarasi `type point = record ... end` membuat scope tersendiri untuk field-fieldnya. `Symbol Table` menunjukkan field `x` dan `y` berada di `lev:1` (`idx 39` dan `40`), sedangkan tipe `point` sendiri di `lev:0` (`idx 41`) dengan `obj:TYPE` dan `ref:1` yang merujuk ke `btab idx 1`, tempat field `record` disimpan. Akses `p.x` dan `p.y` direpresentasikan sebagai `RecordAccessNode`, dengan `tab_index` menunjuk ke entri field yang sesuai dan tipe terselesaikan menjadi `integer`.

Pada penanganan `array`, deklarasi `array` membuat entri di `atab idx 0` dengan `xtyp:INTEGER`, `etyp:INTEGER`, `low:1`, `high:10`, `elsz:1`, `size:10`. Di sini aturan `type checking` yang diuji adalah tipe index `array` tidak boleh `real` dan batas bawah $\leq$ batas atas yang keduanya terpenuhi (`index integer`, $1 \leq 10$). Akses `arr[i]` direpresentasikan sebagai `ArrayAccessNode` dengan tipe elemen `integer`. Variabel `arr` di `tab` memiliki `obj:ARRAY` dan `ref:0` yang merujuk ke `atab idx 0`.

Pengujian ini memvalidasi bahwa ketiga tabel (`tab`, `btab`, `atab`) bekerja bersama secara konsisten untuk tipe komposit, dan bahwa mekanisme `ref` sebagai penghubung antar tabel berfungsi benar. Ini adalah pengujian paling representatif untuk membuktikan kelengkapan `symbol table`.

3.2.7 Pengujian 7

Pengujian ketujuh menguji deklarasi subprogram: satu fungsi (square) dan satu prosedur (showResult), termasuk pemanggilan bersarang `showResult(square(x))`. Program diharapkan lolos tanpa error, dan hasilnya sesuai.

Aspek utama yang diuji adalah manajemen lexical level dan scope. Symbol Table menunjukkan square (FUNCTION) dan showResult (PROCEDURE) berada di `lev:0`, sedangkan parameter masing-masing, yaitu `n` dan `val`, berada di `lev:1`, membuktikan bahwa `open_scope()` dipanggil saat memasuki badan subprogram. Pada `btabs`, blok fungsi square (`idx 1`) dan prosedur showResult (`idx 2`) mencatat `lpar` yang menunjuk parameter terakhir serta `psize:1` yang menandakan satu parameter berukuran satu satuan, yang berbeda dari blok global yang `psize`-nya 0.

Pada fungsi square, statement `square := n * n` valid karena assignment ke nama fungsi merupakan cara mengembalikan nilai, dan tipe hasil (integer) cocok dengan tipe return fungsi. Pemanggilan bersarang `showResult(square(x))` divalidasi berlapis melalui `square(x)` direpresentasikan sebagai `FuncCallExprNode` bertipe integer (tipe return square), lalu nilai integer itu menjadi argumen bagi showResult yang parameternya juga integer, sehingga seluruh rantai pemanggilan konsisten secara tipe.

Pengujian ini memvalidasi bahwa analyzer menangani struktur bersarang dengan benar, membedakan fungsi (punya tipe return) dari prosedur (void), dan mengelola lexical level secara tepat

3.2.8 Pengujian 8

Pengujian kedelapan adalah testcase negatif yang menguji deteksi deklarasi ulang identifier. Program mendeklarasikan `x` bertipe integer, lalu mendeklarasikan kembali `x` bertipe real pada scope yang sama. Keluaran sesuai harapan, yaitu "Semantic Error: Identifier already declared in this scope: 'x'".

Error terdeteksi pada tahap pemrosesan deklarasi, sebelum badan program dijalankan. Sebelum mendaftarkan `x` kedua melalui `enter_tabs`, analyzer memanggil `lookup_in_level` untuk memeriksa duplikat hanya di level aktif. Karena `x` sudah ada di level 0, visitor melempar `SemanticError` jenis `redeclared_error`. Penggunaan `lookup_in_level` (bukan `lookup` biasa) penting agar redeklarasasi di scope yang sama ditolak, sementara shadowing di inner scope tetap diperbolehkan. Perbedaan tipe (integer vs real) tidak relevan dimana yang menjadi pelanggaran adalah nama yang sama di scope yang sama.

Bersama Pengujian 3 (`type_error`) dan Pengujian 5 (`undeclared_error`), pengujian ini melengkapi cakupan ketiga jenis `SemanticError` yang didefinisikan di bagian 2.7, sehingga poin deteksi error pada kesimpulan 4.1 kini terbukti seluruhnya.

Bab IV : Kesimpulan dan Saran

4.1 Kesimpulan

- Semantic analyzer untuk bahasa Arion berhasil diimplementasikan menggunakan Visitor Pattern, mampu melakukan type checking, scope resolution, dan symbol table management.
- Symbol table (tab, btab, atab) berhasil menyimpan informasi seluruh identifier termasuk reserved words dan predefined identifiers.
- Decorated AST berhasil dianotasi dengan type_code, tab_index, dan lev yang akan dibutuhkan pada tahap code generation di Milestone 4.
- Program berhasil mendeteksi berbagai jenis semantic error: type mismatch, undeclared identifier, dan redeclared identifier.

4.2 Saran

- Implementasi error recovery agar semua error dapat dilaporkan sekaligus tanpa berhenti di error pertama.
- Penambahan validasi jumlah dan tipe argumen saat pemanggilan fungsi/prosedur.
- Penambahan informasi nomor baris dan kolom pada pesan error untuk memudahkan debugging.

Lampiran

A. Tautan Github

<https://github.com/tmthyberd/TBS-Tubes-IF2224-2026>

B. Pembagian Tugas

NIM	Deskripsi Tugas	Persentase
13524012	Implementasi sebagian VisitorStatement dan test case, dan fix main program	25%
13524076	Implementasi sebagian VisitorStatement, semantic_main, mengisi laporan bab II	25%
13524078	Implementasi Interface, membuat template dokumen, dan mengisi Bab I	25%
13524092	Implementasi ASTBuilder dan ASTNode, bantu dokumen	25%

C. Referensi

GeeksforGeeks. (2026). *Semantic analysis in compiler design*. Diakses dari <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>

GeeksforGeeks. (2025). *Abstract syntax tree vs parse tree*. Diakses dari <https://www.geeksforgeeks.org/compiler-design/abstract-syntax-tree-vs-parse-tree/>

GeeksforGeeks. (2025). *Symbol table in compiler*. Diakses dari <https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>

GeeksforGeeks. (2025). *Various implementations of symbol table*. Diakses dari <https://www.geeksforgeeks.org/compiler-design/various-implementations-of-symbol-table/>

GeeksforGeeks. (2025). *Type checking in compiler design*. Diakses dari <https://www.geeksforgeeks.org/compiler-design/type-checking-in-compiler-design/>

GeeksforGeeks. (2025). *Syntax directed definition in compiler design*. Diakses dari <https://www.geeksforgeeks.org/compiler-design/compiler-design-syntax-directed-definition/>

GeeksforGeeks. (2025). *Visitor design pattern*. Diakses dari <https://www.geeksforgeeks.org/visitor-design-pattern/>

GeeksforGeeks. (2024). *Visitor method design patterns in C++*. Diakses dari <https://www.geeksforgeeks.org/visitor-method-design-patterns-in-c/>

Wikipedia. (2025). *Abstract syntax tree*. Diakses dari https://en.wikipedia.org/wiki/Abstract_syntax_tree

GeeksforGeeks. (2026). *Scope of variables in C++*. Diakses dari <https://www.geeksforgeeks.org/cpp/scope-of-variables-in-c/>